6 The axiomatic semantics of IMP

In this chapter we turn to the business of systematic verification of programs in **IMP**. The Hoare rules for showing the partial correctness of programs are introduced and shown sound. This involves extending the boolean expressions to a rich language of assertions about program states. The chapter concludes with an example of verification conducted within the framework of Hoare rules.

6.1 The idea

We turn to consider the problem of how to prove that a program we have written in **IMP** does what we require of it.

Let's start with a simple example of a program to compute the sum of the first hundred numbers, the naive way. Here is a program in **IMP** to compute $\sum_{1\leq m\leq 100} m$ (The notation $\sum_{1\leq m\leq 100} m$ means $1+2+\cdots+100$).

$$
\begin{array}{l}
S := 0; \\
N := 1; \\
(\textbf{while } \neg(N = 101) \textbf{ do } S := S + N; N := N + 1)
\end{array}
$$

How would we prove that this program, when it terminates, is such that the value of S is $\sum_{1\leq m\leq 100} m$?

Of course one thing we could do would be to run it according to our operational semantics and see what we get. But suppose we change our program a bit, so that instead of "**while** $\neg(N = 101)$ **do** $\cdots$" we put "**while** $\neg(N = P + 1)$ **do** $\cdots$" and imagine making some arbitrary assignment to P before we begin. In this case the resulting value of S after execution should be $\sum_{1\leq m\leq P} m$, no matter what the value of P. As P can take an infinite set of values we cannot justify this fact simply by running the program for all initial values of P. We need to be a little more clever, and abstract, and use some logic to reason about the program.

We'll end up with a formal proof system for proving properties of **IMP** programs, based on proof rules for each programming construct of **IMP**. Its rules are called Hoare rules or Floyd-Hoare rules. Historically R.W.Floyd invented rules for reasoning about flow charts, and later C.A.R.Hoare modified and extended these to give a treatment of a language like **IMP** but with procedures. Originally their approach was advocated not just for proving properties of programs but also as giving a method for explaining the meaning of program constructs; the meaning of a construct was specified in terms of "axioms" (more accurately rules) saying how to prove properties of it. For this reason, the approach is traditionally called axiomatic semantics.

For now let's not be too formal. Let's look at the program and reason informally about

it, for the moment based on our intuitive understanding of how it behaves. Straightaway we see that the commands $S := 0; N := 1$ initialise the values in the locations. So we can annotate our program with a comment:

$$\begin{aligned}&S := 0; N := 1\\&\{S = 0 \ \wedge \ N = 1\}\\&(\textbf{while } \neg(N = 101) \textbf{ do } S := S + N; N := N + 1)\end{aligned}$$

with the understanding that $S = 0$ for example means the location S has value 0, as in the treatment of boolean expressions. We want a method to justify the final comment in:

$$\begin{aligned}&S := 0; N := 1\\&\{S = 0 \ \wedge \ N = 1\}\\&(\textbf{while } \neg(N = 101) \textbf{ do } S := S + N; N := N + 1)\\&\{S = \sum_{1 \leq m \leq 100} m\}\end{aligned}$$

—meaning that if $S = 0 \ \wedge \ N = 1$ before the execution of the while-loop then $S = \sum_{1 \leq m \leq 100} m$ after its execution.

Looking at the boolean, one fact we know holds after the execution of the while-loop is that we cannot have $N \neq 101$; because if we had $\neg(N = 101)$ then the while-loop would have continued running. So, at the end of its execution we know $N = 101$. But we want to know S!

Of course, with a simple program like this we can look and see what the values of S and N are the first time round the loop, $S = 1, N = 2$. And the second time round the loop $S = 1 + 2, N = 3 \cdots$ and so on, until we see the pattern: after the i th time round the loop $S = 1 + 2 + \cdots + i$ and $N = i + 1$. From which we see, when we exit the loop, that $S = 1 + 2 + \cdots + 100$, because when we exit $N = 101$.

At the beginning and end of each iteration of the while-loop we have

$$S = 1 + 2 + 3 + \cdots + (N - 1) \qquad (I)$$

which expresses the key relationship between the value at location S and the value at location N. The assertion I is called an *invariant* of the while-loop because it remains true under each iteration of the loop. So finally when the loop terminates I will hold at the end. We shall say more about invariants later.

For now it appears we can base a proof system on assertions of the form

$$\{A\}c\{B\}$$

where A and B are assertions like those we've already seen in **Bexp** and c is a command. The precise interpretation of such a compound assertion is this:

> for all states σ which satisfy A if the execution c from state σ terminates in state σ' then σ' satisfies B.

Put another way, $\{A\}c\{B\}$ means that any successful (*i.e.*, terminating) execution of c from a state satisfying A ends up in a state satisfying B. The assertion A is called the precondition and B the postcondition of the partial correctness assertion $\{A\}c\{B\}$.

Assertions of the form $\{A\}c\{B\}$ are called *partial correctness assertions* because they say nothing about the command c if it fails to terminate. As an extreme example consider

$$c \equiv \textbf{while true do skip}.$$

The execution of c from any state does not terminate. According to the interpretation we give above the following partial correctness assertion is valid:

$$\{\textbf{true}\}c\{\textbf{false}\}$$

simply because the execution of c does not terminate. More generally, because c loops, any partial correctness assertion $\{A\}c\{B\}$ is valid. Contrast this with another notion, that of total correctness. Sometimes people write

$$[A]c[B]$$

to mean that the execution of c from any state which satisfies A will terminate in a state which satisfies B. In this book we shall not be concerned much with total correctness assertions.

Warning: There are several different notations around for expressing partial and total correctness. When dipping into a book make doubly sure which notation is used there.

We have left several loose ends. For one, what kinds of assertions A and B do we allow in partial correctness assertions $\{A\}c\{B\}$? We say more in a moment, and turn to a more general issue.

The next issue can be regarded pragmatically as one of notation, though it can be viewed more conceptually as the semantics of assertions for partial correctness—see the "optional" Section 7.5 on denotational semantics using predicate transformers. Firstly let's introduce an abbreviation to mean the state σ satisfies assertion A, or equivalently the assertion A is true at state σ. We abbreviate this to:

$$\sigma \models A.$$

Of course, we'll need to define it, though we all have an intuitive idea of what it means. Consider our interpretation of a partial correctness assertion $\{A\}c\{B\}$. As a command c denotes a partial function from initial states to final states, the partial correctness assertion means:

$$\forall \sigma.\ (\sigma \models A\ \&\ \mathcal{C}[\![c]\!]\sigma \text{ is defined}) \Rightarrow \mathcal{C}[\![c]\!]\sigma \models B.$$

It is awkward working so often with the proviso that $\mathcal{C}[\![c]\!]\sigma$ is defined. Recall Chapter 5 on the denotational semantics of **IMP**. There we suggested that we use the symbol $\perp$ to represent an undefined state (or more strictly, null information about the state). For a command c we can write $\mathcal{C}[\![c]\!]\sigma = \perp$ whenever $\mathcal{C}[\![c]\!]\sigma$ is undefined, and, in accord with the composition of partial functions, take $\mathcal{C}[\![c]\!]\perp = \perp$. If we adopt the convention that $\perp$ satisfies any assertion, then our work on partial correctness becomes much simpler notationally. With the understanding that

$$\perp \models A$$

for any assertion A, we can describe the meaning of $\{A\}c\{B\}$ by

$$\forall \sigma \in \Sigma.\ \sigma \models A \Rightarrow \mathcal{C}[\![c]\!]\sigma \models B.$$

Because we are dealing with partial correctness this convention is consistent with our previous interpretation of partial correctness assertions. It's quite intuitive too; diverging computations denote $\perp$ and as we've seen they satisfy any postcondition.

6.2 The assertion language Assn

What kind of assertions do we wish to make about **IMP** programs? Because we want to reason about boolean expressions we'll certainly need to include all the assertions in **Bexp**. Because we want to make assertions using the quantifiers "$\forall i \cdots$" and "$\exists i \cdots$" we will need to work with extensions of **Bexp** and **Aexp** which include integer variables i over which we can quantify. Then, for example, we can say that an integer k is a multiple of another l by writing

$$\exists i.\ k = i \times l.$$

It will be shown in reasonable detail how to introduce integer variables and quantifiers for a particular language of assertions **Assn**. In principle, everything we'll do with assertions can be done in **Assn**—it is expressive enough—but in examples and exercises we will extend **Assn** in various ways, without being terribly strict about it. (For instance, in one example we'll use the notation $n! = n \times (n-1) \times \cdots \times 2 \times 1$ for the factorial function.)

Firstly, we extend **Aexp** to include integer variables i, j, k, *etc.*. This is done simply by extending the BNF description of **Aexp** by the additional rule which makes any integer variable $i, j, k, \cdots$ an integer expression. So the extended syntactic category **Aexpv** of arithmetic expressions is given by:

$$a ::= n \mid X \mid i \mid a_0 + a_1 \mid a_0 - a_1 \mid a_0 \times a_1$$

where

n ranges over numbers, **N**

X ranges over locations, **Loc**

i ranges over integer variables, **Intvar**.

We extend boolean expressions to include these more general arithmetic expressions and quantifiers, as well as implication. The rules are:

$$A ::= \mathbf{true} \mid \mathbf{false} \mid a_0 = a_1 \mid a_0 \leq a_1 \mid A_0 \wedge A_1 \mid A_0 \vee A_1 \mid \neg A \mid A_0 \Rightarrow A_1 \mid \forall i.A \mid \exists i.A$$

We call the set of extended boolean assertions, **Assn**.

At school we have had experience in manipulating expressions like those above, though in those days we probably wrote mathematics down in a less abbreviated way, not using quantifiers for instance. When we encounter an integer variable i we think of it as standing for some arbitrary integer and do calculations with it like those "unknowns" $x, y, \cdots$ at school. An implication like $A_0 \Rightarrow A_1$ means if A_0 then A_1, and will be true if either A_0 is false or A_1 is true. We have used implication before in our mathematics, and now we have added it to our set of formal assertions **Assn**. We have a "commonsense" understanding of the expressions and assertions (and this should be all that is needed when doing the exercises). However, because we want to reason about proof systems based on assertions, not just examples, we shall be more formal, and give a theory of the meaning of expressions and assertions with integer variables. This is part of the predicate calculus.

6.2.1 Free and bound variables

We say an occurrence of an integer variable i in an assertion is *bound* if it occurs in the scope of an enclosing quantifier $\forall i$ or $\exists i$. If it is not bound we say it is *free*. For example, in

$$\exists i.\ k = i \times l$$

the occurrence of the integer variable i is bound, while those of k and l are free—the variables k and l are understood as standing for particular integers even if we are not

precise about which. The same integer variable can have different occurrences in the same assertion one of which is free and another bound. For example, in

$$(i + 100 \leq 77) \wedge (\forall i.\ j + 1 = i + 3)$$

the first occurrence of i is free and the second bound, while the sole occurrence of j is free.

Although this informal explanation will probably suffice, we can give a formal definition using definition by structural induction. Define the set $\mathrm{FV}(a)$ of free variables of arithmetic expressions, extended by integer variables, $a \in \mathbf{Aexpv}$, by structural induction

$$\begin{aligned}
&\mathrm{FV}(n) = \mathrm{FV}(X) = \emptyset \\
&\mathrm{FV}(i) = \{i\} \\
&\mathrm{FV}(a_0 + a_1) = \mathrm{FV}(a_0 - a_1) = \mathrm{FV}(a_0 \times a_1) = \mathrm{FV}(a_0) \cup \mathrm{FV}(a_1)
\end{aligned}$$

for all $n \in \mathbf{N}, X \in \mathbf{Loc}, i \in \mathbf{Intvar}$, and $a_0, a_1 \in \mathbf{Aexpv}$. Define the free variables $\mathrm{FV}(A)$ of an assertion A by structural induction to be

$$\begin{aligned}
&\mathrm{FV}(\mathbf{true}) = \mathrm{FV}(\mathbf{false}) = \emptyset \\
&\mathrm{FV}(a_0 = a_1) = \mathrm{FV}(a_0 \leq a_1) = \mathrm{FV}(a_0) \cup \mathrm{FV}(a_1) \\
&\mathrm{FV}(A_0 \wedge A_1) = \mathrm{FV}(A_0 \vee A_1) = \mathrm{FV}(A_0 \Rightarrow A_1) = \mathrm{FV}(A_0) \cup \mathrm{FV}(A_1) \\
&\mathrm{FV}(\neg A) = \mathrm{FV}(A) \\
&\mathrm{FV}(\forall i.A) = \mathrm{FV}(\exists i.A) = \mathrm{FV}(A) \setminus \{i\}
\end{aligned}$$

for all $a_0, a_1 \in \mathbf{Aexpv}$, integer variables i and assertions A_0, A_1, A. Thus we have made precise the notion of free variable. Any variable which occurs in an assertion A and yet is not free is said to be bound. An assertion with no free variables is *closed*.

6.2.2 Substitution

We can picture an assertion A as

$$---i---i--$$

say, with free occurrences of the integer variable i. Let a be an arithmetic expression, which for simplicity we assume contains no integer variables. Then

$$A[a/i] \equiv ---a---a--$$

is the result of substituting a for i. If a contained integer variables then it might be necessary to rename some bound variables of A in order to avoid the variables in a becoming bound by quantifiers in A—this is how it's done for general substitutions.

We describe substitution more precisely in the simple case. Let i be an integer variable and a be an arithmetic expression without integer variables, and firstly define substitution into arithmetic expressions by the following structural induction:

$$\begin{aligned}
&n[a/i] \equiv n \qquad X[a/i] \equiv X \\
&j[a/i] \equiv j \qquad i[a/i] \equiv a \\
&(a_0 + a_1)[a/i] \equiv (a_0[a/i] + a_1[a/i]) \\
&(a_0 - a_1)[a/i] \equiv (a_0[a/i] - a_1[a/i]) \\
&(a_0 \times a_1)[a/i] \equiv (a_0[a/i] \times a_1[a/i])
\end{aligned}$$

where n is a number, X a location, j is an integer variable with $j \not\equiv i$ and $a_0, a_1 \in \mathbf{Aexpv}$. Now we define substitution of a for i in assertions by structural induction—remember a does not have any free variables so we need not take any precautions to avoid its variables becoming bound:

$$\begin{aligned}
&\mathbf{true}[a/i] \equiv \mathbf{true} \qquad \mathbf{false}[a/i] \equiv \mathbf{false} \\
&(a_0 = a_1)[a/i] \equiv (a_0[a/i] = a_1[a/i]) \qquad (a_0 \leq a_1)[a/i] \equiv (a_0[a/i] \leq a_1[a/i]) \\
&(A_0 \wedge A_1)[a/i] \equiv (A_0[a/i] \wedge A_1[a/i]) \qquad (A_0 \vee A_1)[a/i] \equiv (A_0[a/i] \vee A_1[a/i]) \\
&(\neg A)[a/i] \equiv \neg(A[a/i]) \qquad (A_0 \Rightarrow A_1)[a/i] \equiv (A_0[a/i] \Rightarrow A_1[a/i]) \\
&(\forall j.A)[a/i] \equiv \forall j.(A[a/i]) \qquad (\forall i.A)[a/i] \equiv \forall i.A \\
&(\exists j.A)[a/i] \equiv \exists j.(A[a/i]) \qquad (\exists i.A)[a/i] \equiv \exists i.A
\end{aligned}$$

where $a_0, a_1 \in \mathbf{Aexpv}$, A_0, A_1 and A are assertions and j is an integer variable with $j \not\equiv i$.

As was mentioned, defining substitution $A[a/i]$ in the case where a contains free variables is awkward because it involves the renaming of bound variables. Fortunately we don't need this more complicated definition of substitution for the moment.

We use the same notation for substitution in place of a location X, so if an assertion $A \equiv ---X ---$ then $A[a/X] = ---a ---$, putting a in place of X. This time the (simpler) formal definition is left to the reader.

Exercise 6.1 Write down an assertion $A \in \mathbf{Assn}$ with one free integer variable i which expresses that i is a prime number, *i.e.* it is required that:

$$\sigma \models^I A \text{ iff } I(i) \text{ is a prime number.}$$

□

Exercise 6.2 Define a formula $LCM \in \mathbf{Assn}$ with free integer variables i, j and k, which means "i is the least common multiple of j and k," *i.e.* it is required that:

$$\sigma \models^I LCM \text{ iff } I(k) \text{ is the least common multiple of } I(i) \text{ and } I(j).$$

(Hint: The least common multiple of two numbers is the smallest non-negative integer divisible by both.) □

6.3 Semantics of assertions

Because arithmetic expressions have been extended to include integer variables, we cannot adequately describe the value of one of these new expressions using the semantic function $\mathcal{A}$ of earlier. We must first interpret integer variables as particular integers. This is the role of interpretations.

An *interpretation* is a function which assigns an integer to each integer variable *i.e.* a function $I : \mathbf{Intvar} \to \mathbf{N}$.

The meaning of expressions, Aexpv

Now we can define a semantic function $\mathcal{A}v$ which gives the value associated with an arithmetic expression with integer variables in a particular state in a particular interpretation; the value of an expression $a \in \mathbf{Aexpv}$ in a an interpretation I and a state σ is written as $\mathcal{A}v[\![a]\!]I\sigma$ or equivalently as $(\mathcal{A}v[\![a]\!](I))(\sigma)$. Define, by structural induction,

$$\begin{aligned}
&\mathcal{A}v[\![n]\!]I\sigma = n\\
&\mathcal{A}v[\![X]\!]I\sigma = \sigma(X)\\
&\mathcal{A}v[\![i]\!]I\sigma = I(i)\\
&\mathcal{A}v[\![a_0 + a_1]\!]I\sigma = \mathcal{A}v[\![a_0]\!]I\sigma + \mathcal{A}v[\![a_1]\!]I\sigma\\
&\mathcal{A}v[\![a_0 - a_1]\!]I\sigma = \mathcal{A}v[\![a_0]\!]I\sigma - \mathcal{A}v[\![a_1]\!]I\sigma\\
&\mathcal{A}v[\![a_0 \times a_1]\!]I\sigma = \mathcal{A}v[\![a_0]\!]I\sigma \times \mathcal{A}v[\![a_1]\!]I\sigma
\end{aligned}$$

The definition of the semantics of arithmetic expressions with integer variables extends the denotational semantics given in Chapter 5 for arithmetic expressions without them.

Proposition 6.3 *For all $a \in$ **Aexp** (without integer variables), for all states σ and for all interpretations I*

$$\mathcal{A}[\![a]\!]\sigma = \mathcal{A}v[\![a]\!]I\sigma.$$

Proof: The proof is a simple exercise in structural induction on arithmetic expressions. □

The meaning of assertions, Assn

Because we include integer variables, the semantic function requires an interpretation function as a further argument. The role of the interpretation function is solely to provide a value in $\mathbf{N}$ which is the interpretation of integer variables.

Notation: We use the notation $I[n/i]$ to mean the interpretation got from interpretation I by changing the value for integer-variable i to n *i.e.*

$$I[n/i](j) = \begin{cases} n & \text{if } j \equiv i, \\ I(j) & \text{otherwise.} \end{cases}$$

We could specify the meanings of assertions in **Assn** in the same way we did for expressions with integer variables, but this time taking the semantic function from assertions to functions which, given an interpretation and state as an argument, returned a truth value. We choose an alternative though equivalent course. Given an interpretation I we define directly those states which satisfy an assertion.

In fact, it is convenient to extend the set of states Σ to the set $\Sigma_\perp$ which includes the value $\perp$ associated with a nonterminating computation—so $\Sigma_\perp =_{def} \Sigma \cup \{\perp\}$. For $A \in \mathbf{Assn}$ we define by structural induction when

$$\sigma \models^I A$$

for a state $\sigma \in \Sigma$, in an interpretation I, and then extend it so $\perp \models^I A$. The relation $\sigma \models^I A$ means state σ *satisfies* A in interpretation I, or equivalently, that assertion A is true at state σ, in interpretation I. By structural induction on assertions, for an interpretation I, we define for all $\sigma \in \Sigma$:

$$\begin{aligned}
&\sigma \models^I \mathbf{true}, \\
&\sigma \models^I (a_0 = a_1) \text{ if } \mathcal{A}v[\![a_0]\!]I\sigma = \mathcal{A}v[\![a_1]\!]I\sigma, \\
&\sigma \models^I (a_0 \leq a_1) \text{ if } \mathcal{A}v[\![a_0]\!]I\sigma \leq \mathcal{A}v[\![a_1]\!]I\sigma, \\
&\sigma \models^I A \wedge B \text{ if } \sigma \models^I A \text{ and } \sigma \models^I B, \\
&\sigma \models^I A \vee B \text{ if } \sigma \models^I A \text{ or } \sigma \models^I B, \\
&\sigma \models^I \neg A \text{ if not } \sigma \models^I A, \\
&\sigma \models^I A \Rightarrow B \text{ if (not } \sigma \models^I A) \text{ or } \sigma \models^I B, \\
&\sigma \models^I \forall i.A \text{ if } \sigma \models^{I[n/i]} A \text{ for all } n \in \mathbf{N}, \\
&\sigma \models^I \exists i.A \text{ if } \sigma \models^{I[n/i]} A \text{ for some } n \in \mathbf{N} \\
&\perp \models^I A.
\end{aligned}$$

Note that, not $\sigma \models^I A$ is generally written as $\sigma \not\models^I A$.

The above tells us formally what it means for an assertion to be true at a state once we decide to interpret integer variables in a particular way fixed by an interpretation. The semantics of boolean expressions provides another way of saying what it means for certain kinds of assertions to be true or false at a state. We had better check that the two ways agree.

Proposition 6.4 *For $b \in$ **Bexp**, $\sigma \in \Sigma$,*

$$\mathcal{B}[\![b]\!]\sigma = \mathbf{true} \text{ iff } \sigma \models^I b, \text{ and}$$
$$\mathcal{B}[\![b]\!]\sigma = \mathbf{false} \text{ iff } \sigma \not\models^I b$$

for any interpretation I.

Proof: The proof is by structural induction on boolean expressions, making use of Proposition 6.3. □

Exercise 6.5 Prove the above proposition. □

Exercise 6.6 Prove by structural induction on expressions $a \in$ **Aexpv** that

$$\mathcal{A}v[\![a]\!]I[n/i]\sigma = \mathcal{A}v[\![a[n/i]]\!]I\sigma.$$

(Note that n occurs as an element of $\mathbf{N}$ on the left and as the corresponding number in $\mathbf{N}$ on the right.)
By using the fact above, prove

$$\sigma \models^I \forall i.A \quad \text{iff} \quad \sigma \models^I A[n/i] \text{ for all } n \in \mathbf{N} \quad \text{and}$$
$$\sigma \models^I \exists i.A \quad \text{iff} \quad \sigma \models^I A[n/i] \text{ for some } n \in \mathbf{N}.$$

□

The extension of an assertion

Let I be an interpretation. Often when establishing properties about assertions and partial correctness assertions it is useful to consider the extension of an assertion with respect to I *i.e.* the set of states at which the assertion is true.

Define the extension of A, an assertion, with respect to an interpretation I to be

$$A^I = \{\sigma \in \Sigma_\perp \mid \sigma \models^I A\}.$$

Partial correctness assertions

A partial correctness assertion has the form

$$\{A\}c\{B\}$$

where $A, B \in \mathbf{Assn}$ and $c \in \mathbf{Com}$. Note that partial correctness assertions are not in **Assn**.

Let I be an interpretation. Let $\sigma \in \Sigma_\perp$. We define the satisfaction relation between states and partial correctness assertions, with respect to I, by

$$\sigma \models^I \{A\}c\{B\} \text{ iff } (\sigma \models^I A \Rightarrow \mathcal{C}[\![c]\!]\sigma \models^I B).$$

for an interpretation I. In other words, a state σ satisfies a partial correctness assertion $\{A\}c\{B\}$, with respect to an interpretation I, iff any successful computation of c from σ ends up in a state satisfying B.

Validity

Let I be an interpretation. Consider $\{A\}c\{B\}$. We are not so much interested in this partial correctness assertion being true at a particular state so much as whether or not it is true at all states *i.e.*

$$\forall \sigma \in \Sigma_\perp.\ \sigma \models^I \{A\}c\{B\},$$

which we can write as

$$\models^I \{A\}c\{B\},$$

expressing that the partial correctness assertion is valid with respect to the interpretation I, because $\{A\}c\{B\}$ is true regardless of which state we consider. Further, consider *e.g.*

$$\{i < X\}X := X + 1\{i < X\}$$

We are not so much interested in the particular value associated with i by the interpretation I. Rather we are interested in whether or not it is true at all states for all interpretations I. This motivates the notion of *validity*. Define

$$\models \{A\}c\{B\}$$

to mean for all interpretations I and all states σ

$$\sigma \models^I \{A\}c\{B\}.$$

When $\models \{A\}c\{B\}$ we say the partial correctness assertion $\{A\}c\{B\}$ is *valid*.

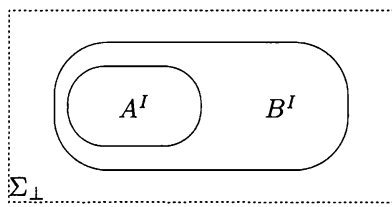
Similarly for any assertion A, write $\models A$ iff for all interpretations I and states σ, $\sigma \models^I A$. Then say A is *valid.*

Warning: Although closely related, our notion of validity is not the same as the notion of validity generally met in a standard course on predicate calculus or "logic programming." There an assertion is called valid iff for all interpretations for operators like $+, x \cdots$, numerals $0, 1, \cdots$, as well as free variables, the assertion turns out to be true. We are not interested in arbitrary interpretations in this general sense because **IMP** programs operate on states based on locations with the standard notions of integer and integer operations. To distinguish the notion of validity here from the more general notion we could call our notion arithmetic-validity, but we'll omit the "arithmetic."

Example: Suppose $\models (A \Rightarrow B)$. Then for any interpretation I,

$$\forall \sigma \in \Sigma.\ ((\sigma \models^I A) \Rightarrow (\sigma \models^I B))$$

i.e. $A^I \subseteq B^I$. In a picture:



So $\models (A \Rightarrow B)$ iff for all interpretations I, all states which satisfy A also satisfy B. □

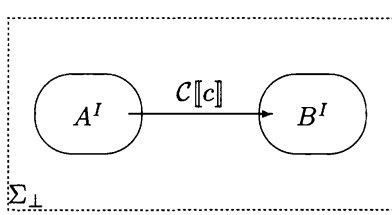
Example: Suppose $\models \{A\}c\{B\}$. Then for any interpretation I,

$$\forall \sigma \in \Sigma.\ ((\sigma \models^I A) \Rightarrow (\mathcal{C}[\![c]\!]\sigma \models^I B)),$$

i.e. the image of A under $\mathcal{C}[\![c]\!]$ is included in B *i.e.*

$$\mathcal{C}[\![c]\!]A^I \subseteq B^I.$$

In a picture:



So $\models \{A\}c\{B\}$ iff for all interpretations I, if c is executed from a state which satisfies A then if its execution terminates in a state that state will satisfy B.[1] □

Exercise 6.7 In an earlier exercise it was asked to write down an assertion $A \in \mathbf{Assn}$ with one free integer variable i expressing that i was prime. By working through the appropriate cases in the definition of the satisfaction relation $\models^I$ between states and assertions, trace out the argument that $\models^I A$ iff $I(i)$ is indeed a prime number. □

6.4 Proof rules for partial correctness

We present proof rules which generate the valid partial correctness assertions. The proof rules are syntax-directed; the rules reduce proving a partial correctness assertion of a compound command to proving partial correctness assertions of its immediate subcommands. The proof rules are often called *Hoare rules* and the proof system, consisting of the collection of rules, *Hoare logic.*

Rule for **skip**:

$$\{A\}\mathbf{skip}\{A\}$$

Rule for assignments:

$$\{B[a/X]\}X := a\{B\}$$

Rule for sequencing:

$$\frac{\{A\}c_0\{C\} \quad \{C\}c_1\{B\}}{\{A\}c_0; c_1\{B\}}$$

Rule for conditionals:

$$\frac{\{A \wedge b\}c_0\{B\} \quad \{A \wedge \neg b\}c_1\{B\}}{\{A\}\mathbf{if}\ b\ \mathbf{then}\ c_0\ \mathbf{else}\ c_1\{B\}}$$

Rule for while loops:

$$\frac{\{A \wedge b\}c\{A\}}{\{A\}\mathbf{while}\ b\ \mathbf{do}\ c\{A \wedge \neg b\}}$$

Rule of consequence:

$$\frac{\models (A \Rightarrow A') \quad \{A'\}c\{B'\} \quad \models (B' \Rightarrow B)}{\{A\}c\{B\}}$$

[1] The picture suggests, incorrectly, that the extensions of assertions A^I and B^I are disjoint; they will both always contain $\perp$, and perhaps have other states in common.

Being rules, there is a notion of derivation for the Hoare rules. In this context the Hoare rules are thought of as a proof system, derivations are called *proofs* and any conclusion of a derivation a *theorem*. We shall write $\vdash \{A\}c\{B\}$ when $\{A\}c\{B\}$ is a theorem.

The rules are fairly easy to understand, with the possible exception of the rules for assignments and while-loops. If an assertion is true of the state before the execution of **skip** it is certainly true afterwards as the state is unchanged. This is the content of the rule for **skip**.

For the moment, to convince that the rule for assignments really is the right way round, it can be tried for a particular assertion such as $X = 3$ for the simple assignment like $X := X + 3$.

The rule for sequential compositions expresses that if $\{A\}c_0\{C\}$ and $\{C\}c_1\{B\}$ are valid then so is $\{A\}c_0; c_1\{B\}$: if a successful execution of c_0 from a state satisfying A ends up in one satisfying C and a successful execution of c_1 from a state satisfying C ends up in one satisfying B, then any successful execution of c_0 followed by c_1 from a state satisfying A ends up in one satisfying B.

The two premises in the rule for conditionals cope with two arms of the conditional.

In the rule for while-loops **while** b **do** c, the assertion A is called the *invariant* because the premise, that $\{A \wedge b\}c\{A\}$ is valid, says that the assertion A is preserved by a full execution of the body of the loop, and in a while loop such executions only take place from states satisfying b. From a state satisfying A either the execution of the while-loop diverges or a finite number of executions of the body are performed, each beginning in a state satisfying b. In the latter case, as A is an invariant the final state satisfies A and also $\neg b$ on exiting the loop.

The consequence rule is peculiar because the premises include valid implications. Any instance of the consequence rule has premises including ones of the form $\models (A \Rightarrow A')$ and $\models (B' \Rightarrow B)$ and so producing an instance of the consequence rule with an eye to applying it in a proof depends on first showing assertions $(A \Rightarrow A')$ and $(B' \Rightarrow B)$ are valid. In general this can be a very hard task—such implications can express complicated facts about arithmetic. Fortunately, because programs often do not involve deep mathematical facts, the demonstration of these validities can frequently be done with elementary mathematics.

6.5 Soundness

We consider for the Hoare rules two very general properties of logical systems:

Soundness: Every rule should preserve validity, in the sense that if the assumptions in the rule's premise is valid then so is its conclusion. When this holds of a rule it is called *sound*. When every rule of a proof system is sound, the proof system itself is said to be *sound*. It follows then by rule-induction that every theorem obtained from the proof system of Hoare rules is a valid partial correctness assertion. (The comments which follow the rules are informal arguments for the soundness of some of the rules.)

Completeness: Naturally we would like the proof system to be strong enough so that all valid partial correctness assertions can be obtained as theorems. We would like the proof system to be *complete* in this sense. (There are some subtle issues here which we discuss in the next chapter.)

The proof of soundness of the rules depends on some facts about substitution.

Lemma 6.8 *Let I be an interpretation. Let $a, a_0 \in$* **Aexpv**. *Let $X \in$* **Loc**. *Then for all interpretations I and states σ*

$$\mathcal{A}v[\![a_0[a/X]]\!]I\sigma = \mathcal{A}v[\![a_0]\!]I\sigma[\mathcal{A}v[\![a]\!]I\sigma/X].$$

Proof: The proof is by structural induction on a_0—exercise! □

Lemma 6.9 *Let I be an interpretation. Let $B \in$* **Assn**, *$X \in$* **Loc** *and $a \in$* **Aexp**. *For all states $\sigma \in \Sigma$*

$$\sigma \models^I B[a/X] \quad \textit{iff} \quad \sigma[\mathcal{A}[\![a]\!]\sigma/X] \models^I B.$$

Proof: The proof is by structural induction on B—exercise! □

Exercise 6.10 Provide the proofs for the lemmas above. □

Theorem 6.11 *Let $\{A\}c\{B\}$ be a partial correctness assertion.*
If $\vdash \{A\}c\{B\}$ then $\models \{A\}c\{B\}$.

Proof: Clearly if we can show each rule is sound (*i.e.* preserves validity in the sense that if its premise consists of valid assertions and partial correctness assertions then so is its conclusion) then by rule-induction we can see that every theorem is valid.

The rule for **skip**: Clearly $\models \{A\}\mathbf{skip}\{A\}$ so the rule for **skip** is sound.

The rule for assignments: Assume $c \equiv (X := a)$. Let I be an interpretation. We have $\sigma \models^I B[a/X]$ iff $\sigma[\mathcal{A}[\![a]\!]\sigma/X] \models^I B$, by Lemma 6.9. Thus

$$\sigma \models^I B[a/X] \Rightarrow \mathcal{C}[\![X := a]\!]\sigma \models^I B,$$

and hence $\models \{B[a/X]\}X := a\{B\}$, showing the soundness of the assignment rule.

The rule for sequencing: Assume $\models \{A\}c\{_1 0C$ and $\models \{C\}c\{_1 1B$. Let I be an interpretation. Suppose $\sigma \models^I A$. Then $\mathcal{C}[\![c_0]\!]\sigma \models^I C$ because $\models^I \{A\}c\{_1 0C$. Also $\mathcal{C}[\![c_1]\!](\mathcal{C}[\![c_0]\!]\sigma) \models_I B$ because $\models^I \{C\}c\{_1 1B$. Hence $\models \{A\}c_0; c_1\{B\}$.

The rule for conditionals: Assume $\models \{A \wedge b\}c_0\{B\}$ and $\models \{A \wedge \neg b\}c_1\{B\}$. Let I be an interpretation. Suppose $\sigma \models_I A$. Either $\sigma \models_I b$ or $\sigma \models_I \neg b$. In the former case $\sigma \models_I A \wedge b$ so $\mathcal{C}[\![c_0]\!]\sigma \models^I B$, as $\models^I \{A \wedge b\}c_0\{B\}$. In the latter case $\sigma \models_I A \wedge \neg b$ so $\mathcal{C}[\![c_1]\!]\sigma \models^I B$, as $\models^I \{A \wedge \neg b\}c_1\{B\}$. This ensures $\models \{A\}$**if** b **then** c_0 **else** $c_1\{B\}$.

The rule for while-loops: Assume $\models \{A \wedge b\}c\{A\}$, *i.e.* A is an invariant of

$$w \equiv \textbf{while } b \textbf{ do } c.$$

Let I be an interpretation. Recall that $\mathcal{C}[\![w]\!] = \bigcup_{n\in\omega} \theta_n$ where

$$\theta_0 = \emptyset,$$
$$\theta_{n+1} = \{(\sigma, \sigma') \mid \mathcal{B}[\![b]\!]\sigma = \textbf{true} \ \&\ (\sigma, \sigma') \in \theta_n \circ \mathcal{C}[\![c]\!]\} \cup \{(\sigma, \sigma) \mid \mathcal{B}[\![b]\!]\sigma = \textbf{false}.\}$$

We shall show by mathematical induction that $P(n)$ holds where

$$P(n) \iff_{def} \forall \sigma, \sigma' \in \Sigma.\ (\sigma, \sigma') \in \theta_n \ \&\ \sigma \models^I A \ \Rightarrow\ \sigma' \models^I A \wedge \neg b$$

for all $n \in \omega$. It then follows that

$$\sigma \models^I A \Rightarrow \mathcal{C}[\![w]\!]\sigma \models^I A \wedge \neg b$$

for all states σ, and hence that $\models \{A\}w\{A \wedge \neg b\}$, as required.
Base case $n = 0$: When $n = 0$, $\theta_0 = \emptyset$ so that induction hypothesis $P(0)$ is vacuously true.
Induction Step: We assume the induction hypothesis $P(n)$ holds for $n \geq 0$ and attempt to prove $P(n+1)$. Suppose $(\sigma, \sigma') \in \theta_{n+1}$ and $\sigma \models^I A$. Either
(i) $\mathcal{B}[\![b]\!]\sigma = \textbf{true}$ and $(\sigma, \sigma') \in \theta_n \circ \mathcal{C}[\![c]\!]$, or
(ii) $\mathcal{B}[\![b]\!]\sigma = \textbf{false}$ and $\sigma' = \sigma$.

We show in either case that $\sigma' \models^I A \wedge \neg b$.
Assume (i). As $\mathcal{B}[\![b]\!]\sigma = \mathbf{true}$ we have $\sigma \models^I b$ and hence $\sigma \models^I A \wedge b$. Also $(\sigma, \sigma'') \in \mathcal{C}[\![c]\!]$ and $(\sigma'', \sigma') \in \theta_n$ for some state σ''. We obtain $\sigma'' \models^I A$, as $\models \{A \wedge b\}c\{A\}$. From the assumption $P(n)$, we obtain $\sigma' \models^I A \wedge \neg b$.
Assume (ii). As $\mathcal{B}[\![b]\!]\sigma = \mathbf{false}$ we have $\sigma \models^I \neg b$ and hence $\sigma \models^I A \wedge \neg b$. But $\sigma' = \sigma$.

This establishes the induction hypothesis $P(n+1)$. By mathematical induction we conclude $P(n)$ holds for all n. Hence the rule for while loops is sound.

The consequence rule: Assume $\models (A \Rightarrow A')$ and $\models \{A'\}c\{B'\}$ and $\models (B' \Rightarrow B)$. Let I be an interpretation. Suppose $\sigma \models^I A$. Then $\sigma \models^I A'$, hence $\mathcal{C}[\![c]\!]\sigma \models^I B'$ and hence $\mathcal{C}[\![c]\!]\sigma \models^I B$. Thus $\models \{A\}c\{B\}$. The consequence rule is sound.

By rule-induction, every theorem is valid. □

Exercise 6.12 Prove the above using only the operational semantics, instead of the denotational semantics. What proof method is used for the case of while-loops? □

6.6 Using the Hoare rules—an example

The Hoare rules determine a notion of formal proof of partial correctness assertions through the idea of derivation. This is useful in the mechanisation of proofs. But in practice, as human beings faced with the task of verifying a program, we need not be so strict and can argue at a more informal level when using the Hoare rules. (Indeed working with the more formal notion of derivation might well distract from getting the proof; the task of producing the formal derivation should be delegated to a proof assistant like LCF or HOL [74], [43].)

As an example we show in detail how to use the Hoare rules to verify that the command

$$w \equiv (\mathbf{while}\ X > 0\ \mathbf{do}\ Y := X \times Y; X := X - 1)$$

does indeed compute the factorial function $n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$, with $0!$ understood to be 1, given that $X = n$, a nonnegative number, and $Y = 1$ initially.[2]

More precisely, we wish to prove:

$$\{X = n \wedge n \geq 0 \wedge Y = 1\}w\{Y = n!\}.$$

To prove this we must clearly invoke the proof rule for while-loops which requires an invariant. Take

$$I \equiv (Y \times X! = n! \wedge X \geq 0).$$

[2]For this example, we imagine our syntax of programs and assertions to be extended to include $>$ and the factorial function which strictly speaking do not appear in the boolean and arithmetic expressions defined earlier.

We show I is indeed an invariant *i.e.*

$$\{I \wedge X > 0\}Y := X \times Y; X := X - 1\{I\}.$$

From the rule for assignment we have

$$\{I[(X-1)/X]\}X := X - 1\{I\}$$

where $I[(X-1)/X] \equiv (Y \times (X-1)! = n! \wedge (X-1) \geq 0)$. Again by the assignment rule:

$$\{X \times Y \times (X-1)! = n! \wedge (X-1) \geq 0\}Y := X \times Y\{I[(X-1)/X]\}.$$

Thus, by the rule for sequencing,

$$\{X \times Y \times (X-1)! = n! \wedge (X-1) \geq 0\}Y := X \times Y; X := (X-1)\{I\}.$$

Clearly

$$\begin{aligned} I \wedge X > 0 &\Rightarrow Y \times X! = n! \wedge X \geq 0 \wedge X > 0 \\ &\Rightarrow Y \times X! = n! \wedge X \geq 1 \\ &\Rightarrow X \times Y \times (X-1)! = n! \wedge (X-1) \geq 0. \end{aligned}$$

Thus by the consequence rule

$$\{I \wedge X > 0\}Y := X \times Y; X := (X-1)\{I\}$$

establishing that I is an invariant.

Now applying the rule for while-loops we obtain

$$\{I\}w\{I \ \wedge \ X \not> 0\}.$$

Clearly $(X = n) \wedge (n \geq 0) \wedge (Y = 1) \Rightarrow I$, and

$$\begin{aligned} I \ \wedge \ X \not> 0 &\Rightarrow Y \times X! = n! \wedge X \geq 0 \wedge X \not> 0 \qquad (*) \\ &\Rightarrow Y \times X! = n! \wedge X = 0 \\ &\Rightarrow Y \times 0! = Y = n! \end{aligned}$$

Thus by the consequence rule we conclude

$$\{(X = n) \wedge (Y = 1)\}w\{Y = n!\}.$$

There are a couple of points to note about the proof given in the example. Firstly, in dealing with a chain of commands composed in sequence it is generally easier to proceed

in a right-to-left manner because the rule for assignment is of this nature. Secondly, our choice of I may seem unduly strong. Why did we include the assertion $X \geq 0$ in the invariant? Notice where it was used, at $(*)$, and without it we could not have deduced that on exiting the while-loop the value of X is 0. In getting invariants to prove what we want they often must be strengthened. They are like induction hypotheses. One obvious way to strengthen an invariant is to specify the range of the variables and values at the locations as tightly as possible. Undoubtedly, a common difficulty in examples is to get stuck on proving the "exit conditions". In this case, it is a good idea to see how to strengthen the invariant with information about the variables and locations in the boolean expression.

Thus it is fairly involved to show even trivial programs are correct. The same is true, of course, for trivial bits of mathematics, too, if one spells out all the details in a formal proof system. One point of formal proof systems is that proofs of properties of programs can be automated as in *e.g.*[74][41]—see also Section 7.4 on verification conditions in the next chapter. There is another method of application of such formal proof systems which has been advocated by Dijkstra and Gries among others, and that is to use the ideas in the study of program correctness in the design and development of programs. In his book "The Science of Programming" [44], Gries says

> "the study of program correctness proofs has led to the discovery and elucidation of methods for developing programs. Basically, one attempts to develop a program and its proof hand-in-hand, with the proof ideas leading the way!"

See Gries' book for many interesting examples of this approach.

Exercise 6.13 Prove, using the Hoare rules, the correctness of the partial correctness assertion:

$$\begin{array}{l}\{1 \leq N\} \\ P := 0; \\ C := 1; \\ (\textbf{while } C \leq N \textbf{ do } P := P + M; C := C + 1) \\ \{P = M \times N\}\end{array}$$

□

Exercise 6.14 Find an appropriate invariant to use in the while-rule for proving the following partial correctness assertion:

$$\{i = Y\}\textbf{while } \neg(Y = 0) \textbf{ do } Y := Y - 1; X := 2 \times X\{X = 2^i\}$$

□

Exercise 6.15 Using the Hoare rules, prove that for integers n, m,

$$\{X = m \wedge Y = n \wedge Z = 1\}c\{Z = m^n\}$$

where c is the while-program

$$\begin{array}{l}\textbf{while } \neg(Y = 0) \textbf{ do} \\ ((\textbf{while } \mathit{even}(Y) \textbf{ do } X := X \times X; Y := Y/2); \\ Z := Z \times X; Y := Y - 1)\end{array}$$

with the understanding that $Y/2$ is the integer resulting from dividing the contents of Y by 2, and $\mathit{even}(Y)$ means the content of Y is an even number.
(Hint: Use $m^n = Z \times X^Y$ as the invariants.) □

Exercise 6.16
(i) Show that the greatest common divisor, $\gcd(n, m)$ of two positive numbers n, m satisfies:

$$\begin{array}{l}(a)\ n > m \Rightarrow \gcd(n, m) = \gcd(n - m, m) \\ (b)\ \gcd(n, m) = \gcd(m, n) \\ (c)\ \gcd(n, n) = n.\end{array}$$

(ii) Using the Hoare rules prove

$$\{N = n \wedge M = m \wedge 1 \leq n \wedge 1 \leq m\}\text{Euclid}\{X = \gcd(n, m)\}$$

where

$$\begin{array}{rl}\text{Euclid} \equiv & \textbf{while } \neg(M = N) \textbf{ do} \\ & \quad \textbf{if } M \leq N \\ & \quad \textbf{then } N := N - M \\ & \quad \textbf{else } M := M - N.\end{array}$$

□

Exercise 6.17 Provide a Hoare rule for the repeat construct and prove it sound.
(*cf.* Exercise 5.9.) □

6.7 Further reading

The book [44] by Gries has already been mentioned. Dijkstra's "A discipline of programming" [36] has been very influential. A more elementary book in the same vein

is Backhouse's "Program construction and verification" [12]. A recent book which is recommended is Cohen's "Programming in the 1990's" [32]. A good book with many exercises is Alagić and Arbib's "The design of well-structured and correct programs" [5]. An elementary treatment of Hoare logic with a lot of informative discussion can be found in Gordon's recent book [42]. Alternatives to this book's treatment, concentrating more on semantic issues than the other references, can be found in de Bakker's "Mathematical theory of program correctness" [13] and Loeckx and Sieber's "The foundations of program verification" [58].

7 Completeness of the Hoare rules

In this chapter it is discussed what it means for the Hoare rules to be complete. Gödel's Incompleteness Theorem implies there is no complete proof system for establishing precisely the valid assertions. The Hoare rules inherit this incompleteness. However by separating incompleteness of the assertion language from incompleteness due to inadequacies in the axioms and rules for the programming language constructs, we can obtain relative completeness in the sense of Cook. The proof that the Hoare rules are relatively complete relies on the idea of weakest liberal precondition, and leads into a discussion of verification-condition generators.

7.1 Gödel's Incompleteness Theorem

Look again at the proof rules for partial correctness assertions, and in particular at the consequence rule. Knowing we have a rule instance of the consequence rule requires that we determine that certain assertions in **Assn** are valid. Ideally, of course, we would like a proof system of axioms and rules for assertions which enabled us to prove all the assertions of **Assn** which are valid, and none which are invalid. Naturally we would like the proof system to be *effective* in the sense that it is a routine matter to check that something proposed as a rule instance really is one. It should be routine in the sense that there is a computable method in the form of a program which, with input a real rule instance, returns a confirmation that it is, and returns no confirmation on inputs which are not rule instances, without necessarily even terminating. Lacking such a computable method we might well have a proof derivation without knowing it because it uses a step we cannot check is a rule instance. We cannot claim that the proof system of Hoare rules is effective because we do not have a computable method for checking instances of the consequence rule. Having such depends on having a computable method to check that assertions of **Assn** are valid. But here we meet an absolute limit. The great Austrian logician Kurt Gödel showed that it is logically impossible to have an effective proof system in which one can prove precisely the valid assertions of **Assn**. This remarkable result, called Gödel's Incompleteness Theorem[1] is not so hard to prove nowadays, if one goes about it via results from the theory of computability. Indeed a proof of the theorem, stated now, will be given in Section 7.3 based on some results from computability. Any gaps or shortcomings there can be made up for by consulting the Appendix on computability and undecidability based on the language of while programs, **IMP**.

[1] The Incompleteness Theorem is not to be confused with Gödel's Completeness Theorem which says that the proof system for predicate calculus generates precisely those assertions which are valid for *all* interpretations.

Theorem 7.1 *Gödel's Incompleteness Theorem (1931):*
There is no effective proof system for **Assn** *such that the theorems coincide with the valid assertions of* **Assn**.

This theorem means we cannot have an effective proof system for partial correctness assertions. As $\models B$ iff $\models \{\mathbf{true}\}\mathbf{skip}\{B\}$, if we had an effective proof system for partial correctness it would reduce to an effective proof system for assertions in **Assn**, which is impossible by Gödel's Incompleteness Theorem. In fact we can show there is no effective proof system for partial correctness assertions more directly.

Proposition 7.2 *There is no effective proof system for partial correctness assertions such that its theorems are precisely the valid partial correctness assertions.*

Proof: Observe that $\models \{\mathbf{true}\}c\{\mathbf{false}\}$ iff the command c diverges on all states. If we had an effective proof system for partial correction assertions it would yield a computable method of confirming that a command c diverges on all states. But this is known to be impossible—see Exercise A.13 of the Appendix. □

Faced with this unsurmountable fact, we settle for the proof system of Hoare rules in Section 6.4 even though we know it to be not effective because of the nature of the consequence rule; determining that we have an instance of the consequence rule is dependent on certain assertions being valid. Still, we can inquire as to the completeness of this system. That it is complete was established by S. Cook in [33]. If a partial correctness assertion is valid then there is a proof of it using the Hoare rules, *i.e.* for any partial correctness assertion $\{A\}c\{B\}$,

$$\models \{A\}c\{B\} \text{ implies } \vdash \{A\}c\{B\},$$

though the fact that it is a proof can rest on certain assertions in **Assn** being valid. It is as if in building proofs one could consult an oracle at any stage one needs to know if an assertion in **Assn** is valid. For this reason Cook's result is said to establish the *relative completeness* of the Hoare rules for partial correctness—their completeness is relative to being able to draw from the set of valid assertions about arithmetic. In this way one tries to separate concerns about programs and reasoning about them from concerns to do with arithmetic and the incompleteness of any proof system for it.

7.2 Weakest preconditions and expressiveness

The proof of relative completeness relies on another concept. Consider trying to prove

$$\{A\}c_0; c_1\{B\}.$$

In order to use the rule for composition one requires an intermediate assertion C so that

$$\{A\}c_0\{C\} \text{ and } \{C\}c_1\{B\}$$

are provable. How do we know such an intermediate assertion C can be found? A sufficient condition is that for every command c and postconditions B we can express their weakest precondition[2] in **Assn**.

Let $c \in \mathbf{Com}$ and $B \in \mathbf{Assn}$. Let I be an interpretation. The weakest precondition $wp^I[\![c, B]\!]$ of B with respect to c in I is defined by:

$$wp^I[\![c, B]\!] = \{\sigma \in \Sigma_\perp \mid C[\![c]\!]\sigma \models^I B\}.$$

It's all those states from which the execution of c either diverges or ends up in a final state satisfying B. Thus if $\models^I \{A\}c\{B\}$ we know

$$A^I \subseteq wp^I[\![c, B]\!]$$

and *vice versa*. Thus $\models^I \{A\}c\{B\}$ iff $A^I \subseteq wp^I[\![c, B]\!]$.

Suppose there is an assertion A_0 such that in all interpretations I,

$$A_0^I = wp^I[\![c, B]\!].$$

Then

$$\models^I \{A\}c\{B\} \text{ iff } \models^I (A \Rightarrow A_0),$$

for any interpretation I *i.e.*

$$\models \{A\}c\{B\} \text{ iff } \models (A \Rightarrow A_0).$$

So we see why it is called the weakest precondition, it is implied by any precondition which makes the partial correctness assertion valid. However it's not obvious that a particular language of assertions has an assertion A_0 such that $A_0^I = wp^I[\![c, B]\!]$.

Definition: Say **Assn** is *expressive* iff for every command c and assertion B there is an assertion A_0 such that $A_0^I = wp^I[\![c, B]\!]$ for any interpretation I.

In showing expressiveness we will use Gödel's β predicate to encode facts about sequences of states as assertions in **Assn**. The β predicate involves the operation a **mod** b which gives the remainder of a when divided by b. We can express this notion as an assertion in **Assn**. For $x = a$ **mod** b we write

[2]What we shall call weakest precondition is generally called weakest liberal precondition, the term weakest precondition referring to a related notion but for total correctness.

$$a \geq 0 \quad \wedge \quad b \geq 0 \quad \wedge$$
$$\exists k.[k \geq 0 \ \wedge \ k \times b \leq a \ \wedge \ (k+1) \times b > a \ \wedge \ x = a - (k \times b)].$$

Lemma 7.3 *Let $\beta(a, b, i, x)$ be the predicate over natural numbers defined by*

$$\beta(a, b, i, x) \Leftrightarrow_{def} x = a \ \mathbf{mod}(1 + (1 + i) \times b).$$

For any sequence $n_0, \ldots, n_k$ of natural numbers there are natural numbers n, m such that for all j, $0 \leq j \leq k$, and all x we have

$$\beta(n, m, j, x) \Leftrightarrow x = n_j.$$

Proof: The proof of this arithmetical fact is left to the reader as a small series of exercises at the end of this section. □

The β predicate is important because with it we can encode a sequence of k natural numbers $n_0, \cdots, n_k$ as a pair n, m. Given n, m, for any length k, we can extract a sequence, *viz.*that sequence of numbers $n_0, \cdots, n_k$ such that

$$\beta(n, m, j, n_j)$$

for $0 \leq j \leq k$. Notice that the definition of β shows that the list $n_0, \cdots, n_k$ is uniquely determined by the choice of n, m. The lemma above asserts that any sequence $n_0, \cdots, n_k$ can be encoded in this way.

We must now face a slight irritation. Our states and our language of assertions can involve negative as well as positive numbers. We are obliged to extend Gödel's β predicate so as to encode sequences of positive and negative numbers. Fortunately, this is easily done by encoding positive numbers as the even and negative numbers as the odd natural numbers.

Lemma 7.4 *Let $F(x, y)$ be the predicate over natural numbers x and positive and negative numbers y given by*

$$\begin{array}{lll} F(x,y) & \equiv & x \geq 0 \ \ \& \\ \exists z \geq 0. & & [(x = 2 \times z \Rightarrow y = z) \ \ \& \\ & & (x = 2 \times z + 1 \Rightarrow y = -z)] \end{array}$$

Define

$$\beta^{\pm}(n, m, j, y) \Leftrightarrow_{def} \exists x.(\beta(n, m, j, x) \wedge F(x, y)).$$

Then for any sequence $n_0, \ldots, n_k$ of positive or negative numbers there are natural numbers n, m such that for all j, $0 \leq j \leq k$, and all x we have

$$\beta^{\pm}(n, m, j, x) \Leftrightarrow x = n_j.$$

Proof: Clearly $F(n, m)$ expresses the 1-1 correspondence between natural numbers $m \in \omega$ and $n \in \mathbf{N}$ in which even m stand for non-negative and odd m for negative numbers. The lemma follows from Lemma 7.3. □

The predicate $\beta^{\pm}$ is expressible in **Assn** because β and F are. To avoid introducing a further symbol, let us write $\beta^{\pm}$ for the assertion in **Assn** expressing this predicate. This assertion in **Assn** will have free integer variables, say n, m, j, x, understood in the same way as above, *i.e.* n, m encodes a sequence with jth element x. We will want to use other integer variables besides n, m, j, x, so we write $\beta^{\pm}(n', m', j', x')$ as an abbreviation for $\beta^{\pm}[n'/n, m'/m, j'/j, x'/x]$, got by substituting the the integer variable n' for n, m' for m, and so on. We have not give a formal definition of what it means to substitute integer variables in an assertion. The definition of substitution in Section 6.2.2 only defines substitutions $A[a/i]$ of arithmetic expressions a without integer variables, for an integer variable i in an assertion A. However, as long as the variables n', m', j', x' are "fresh" in the sense of their being distinct and not occurring (free or bound) in $\beta^{\pm}$, the same definition applies equally well to the substitution of integer variables; the assertion $\beta^{\pm}[n'/n, m'/m, j'/j, x'/x]$ is that given by $\beta^{\pm}[n'/n][m'/m][j'/j][x'/x]$ using the definition of Section 6.2.2.[3]

Now we can show:

Theorem 7.5 Assn *is expressive.*

Proof: We show by structural induction on commands c that for all assertions B there is an assertion $w[\![c, B]\!]$ such that for all interpretations I

$$wp^I[\![c, B]\!] = w[\![c, B]\!]^I,$$

for all commands c.

Note that by the definition of weakest precondition that, for I an interpretation, the equality $wp^I[\![c, B]\!] = w[\![c, B]\!]^I$ amounts to

$$\sigma \models^I w[\![c, B]\!] \text{ iff } \mathcal{C}[\![c]\!]\sigma \models^I B,$$

[3] To illustrate the technical problem with substitution of integer variables which are not fresh, consider the assertion $A \equiv (\exists i'.\ 2 \times i' = i)$ which means "i is even." The naive definition of $A[i'/i]$ yields the assertion $(\exists i'.\ 2 \times i' = i')$ which happens to be valid, and so certainly does not mean "i is even."

holding for all states σ, a fact we shall use occasionally in the proof.

$c \equiv \textbf{skip}$: In this case, take $w[\![\textbf{skip}, B]\!] \equiv B$. Clearly, for all states σ and interpretations I,

$$\begin{aligned}\sigma \in wp^I[\![\textbf{skip}, B]\!] \text{ iff } & \mathcal{C}[\![\textbf{skip}]\!]\sigma \models^I B \\ \text{iff } & \sigma \models^I B \\ \text{iff } & \sigma \models^I w[\![\textbf{skip}, B]\!].\end{aligned}$$

$c \equiv (X := a)$: In this case, define $w[\![X := a, B]\!] \equiv B[a/X]$. Then

$$\begin{aligned}\sigma \in wp^I[\![X := a, B]\!] \text{ iff } & \sigma[\mathcal{A}[\![a]\!]\sigma/X] \models^I B \\ \text{iff } & \sigma \models^I B[a/X] \qquad \text{by Lemma 6.9} \\ \text{iff } & \sigma \models^I w[\![X := a, B]\!].\end{aligned}$$

$c \equiv c_0; c_1$: Inductively define $w[\![c_0; c_1, B]\!] \equiv w[\![c_0, w[\![c_1, B]\!]]\!]$. Then, for $\sigma \in \Sigma$ and interpretation I,

$$\begin{aligned}\sigma \in wp^I[\![c_0; c_1, B]\!] \text{ iff } & \mathcal{C}[\![c_0; c_1]\!]\sigma \models^I B \\ \text{iff } & \mathcal{C}[\![c_1]\!](\mathcal{C}[\![c_0]\!]\sigma) \models^I B \\ \text{iff } & \mathcal{C}[\![c_0]\!]\sigma \models^I w[\![c_1, B]\!], \qquad \text{by induction,} \\ \text{iff } & \sigma \models^I w[\![c_0, w[\![c_1, B]\!]]\!], \qquad \text{by induction,} \\ \text{iff } & \sigma \models^I w[\![c_0; c_1, B]\!].\end{aligned}$$

$c \equiv \textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1$: Define

$$w[\![\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, B]\!] \equiv [(b \wedge w[\![c_0, B]\!]) \vee (\neg b \wedge w[\![c_1, B]\!])].$$

Then, for $\sigma \in \Sigma$ and interpretation I,

$$\begin{aligned}\sigma \in wp^I[\![c, B]\!] \text{ iff } & \mathcal{C}[\![c]\!]\sigma \models^I B \\ \text{iff } & ([\mathcal{B}[\![b]\!]\sigma = \textbf{true} \ \&\ \mathcal{C}[\![c_0]\!]\sigma \models^I B] \text{ or} \\ & \quad [\mathcal{B}[\![b]\!]\sigma = \textbf{false} \ \&\ \mathcal{C}[\![c_1]\!]\sigma \models^I B]) \\ \text{iff } & ([\sigma \models^I b \ \&\ \sigma \models^I w[\![c_0, B]\!]] \text{ or} \\ & \quad [\sigma \models^I \neg b \ \&\ \sigma \models^I w[\![c_1, B]\!]]), \quad \text{by induction,} \\ \text{iff } & \sigma \models^I [(b \wedge w[\![c_0, B]\!]) \vee (\neg b \wedge w[\![c_1, B]\!])] \\ \text{iff } & \sigma \models^I w[\![c, B]\!].\end{aligned}$$

$c \equiv$ **while** b **do** c_0: This is the one difficult case. For a state σ and interpretation I, we have (from Exercise 5.8) that $\sigma \in wp^I[\![c, B]\!]$ iff

$$\begin{array}{l} \forall k\, \forall \sigma_0, \ldots, \sigma_k \in \Sigma. \\ [\sigma = \sigma_0\ \& \\ \\ \forall i (0 \leq i < k).\ (\ \sigma_i \models^I b\ \& \\ \\ \qquad\qquad \mathcal{C}[\![c_0]\!]\sigma_i = \sigma_{i+1})] \\ \\ \Rightarrow (\sigma_k \models^I b \vee B). \end{array} \tag{1}$$

As it stands the mathematical characterisation of states σ in $wp^I[\![c, B]\!]$ is not an assertion in **Assn**; in particular it refers directly to states $\sigma_0, \cdots, \sigma_k$. However we show how to replace it by an equivalent description which is. The first step is to replace all references to the states $\sigma_0, \ldots, \sigma_k$ by references to the values they contain at the locations mentioned in c and B. Suppose $\bar{X} = X_1, \ldots, X_l$ are the locations mentioned in c and B—the values at the remaining locations are irrelevant to the computation. We make use of the following fact:
Suppose A is an assertion in **Assn** which mentions only locations from $\bar{X} = X_1, \ldots, X_l$. For a state σ, let $s_i = \sigma(X_i)$, for $1 \leq i \leq l$, and write $\bar{s} = s_1, \cdots, s_l$. Then

$$\sigma \models^I A \text{ iff } \models^I A[\bar{s}/\bar{X}] \tag{$*$}$$

for any interpretation I. The assertion $A[\bar{s}/\bar{X}]$ is that obtained by the simultaneous substitution of $\bar{s}$ for $\bar{X}$ in A. This fact can be proved by structural induction (Exercise!).

Using the fact $(*)$ we can convert (1) into an equivalent assertion about sequences. For $i \geq 0$, let $\bar{s_i}$ abbreviate $s_{i1}, \ldots, s_{il}$, a sequence in $\mathbf{N}$. We claim: $\sigma \in wp^I[\![c, B]\!]$ iff

$$\begin{array}{l} \forall k\, \forall \bar{s}_0, \ldots, \bar{s}_k \in \mathbf{N}. \\ [\sigma \models^I \bar{X} = \bar{s}_0\ \& \\ \\ \forall i\ (0 \leq i < k).\ (\models^I b[\bar{s}_i/\bar{X}]\ \& \\ \\ \qquad\qquad \models^I (w[\![c_0, \bar{X} = \bar{s}_{i+1}]\!] \wedge \neg w[\![c_0, \mathbf{false}]\!])[\bar{s}_i/\bar{X}]) \\ \\ \Rightarrow \models^I (b \vee B)[\bar{s}_k/\bar{X}] \end{array} \tag{2}$$

We have used $\bar{X} = \bar{s}_0$ to abbreviate $X_1 = s_{01} \wedge \ldots \wedge X_l = s_{0l}$.

To prove the claim we argue that (1) and (2) are equivalent. Parts of the argument are straightforward. For example, it follows directly from (*) that, assuming state σ_i has values $\bar{s}_i$ at $\bar{X}$,

$$\sigma_i \models^I b \text{ iff } \models^I b[\bar{s}_i/\bar{X}],$$

for an interpretation I. The hard part hinges on showing that assuming σ_i and σ_{i+1} have values $\bar{s}_i$ and $\bar{s}_{i+1}$, respectively, at $\bar{X}$ and agree elsewhere, we have

$$\mathcal{C}[\![c_0]\!]\sigma_i = \sigma_{i+1} \text{ iff } \models^I (w[\![c_0, \bar{X} = \bar{s}_{i+1}]\!] \wedge \neg w[\![c_0, \mathbf{false}]\!])[\bar{s}_i/\bar{X}],$$

for an interpretation I. To see this we first observe that

$$\mathcal{C}[\![c_0]\!]\sigma_i = \sigma_{i+1} \text{ iff } \sigma_i \in wp^I[\![c_0, \bar{X} = \bar{s}_{i+1}]\!] \;\&\; \mathcal{C}[\![c_0]\!]\sigma_i \text{ is defined.}$$

(Why?) From the induction hypothesis we obtain

$$\begin{array}{lll} \sigma_i \in wp^I[\![c_0, \bar{X} = \bar{s}_{i+1}]\!] & \text{iff} & \sigma_i \models^I (w[\![c_0, \bar{X} = \bar{s}_{i+1}]\!], \text{ and} \\ \mathcal{C}[\![c_0]\!]\sigma_i \text{ is defined} & \text{iff} & \sigma_i \models^I \neg w[\![c_0, \mathbf{false}]\!] \end{array}$$

—recall that $\sigma_i \in wp^I[\![c_0, \mathbf{false}]\!]$ iff c_0 diverges on σ_i. Consequently,

$$\begin{array}{lll} \mathcal{C}[\![c_0]\!]\sigma_i = \sigma_{i+1} \text{ iff } & \sigma_i \models^I (w[\![c_0, \bar{X} = \bar{s}_{i+1}]\!] \wedge \neg w[\![c_0, \mathbf{false}]\!]) & \\ \text{iff } & \models^I (w[\![c_0, \bar{X} = \bar{s}_{i+1}]\!] \wedge \neg w[\![c_0, \mathbf{false}]\!])[\bar{s}_i/\bar{X}] & \text{by (*).} \end{array}$$

This covers the difficulties in showing (1) and (2) equivalent.

Finally, notice how (2) can be expressed in **Assn**, using the Gödel predicate $\beta^{\pm}$. For simplicity assume $l = 1$ with $\bar{X} = X$. Then we can rephrase (2) to get: $\sigma \in wp^I[\![c, B]\!]$ iff

$$\begin{array}{l} \sigma \models^I \forall k\, \forall m, n \geq 0. \\ \quad [\beta^{\pm}(n, m, 0, X) \wedge \\ \quad \forall i\, (0 \leq i < k).\, (\forall x.\, \beta^{\pm}(n, m, i, x) \Rightarrow b[x/X]) \wedge \\ \qquad \forall x, y.\, (\beta^{\pm}(n, m, i, x) \wedge \beta^{\pm}(n, m, i+1, y) \Rightarrow \\ \qquad\qquad (w[\![c_0, X = y]\!] \wedge \neg w[\![c_0, \mathbf{false}]\!])[x/X])] \\ \qquad \Rightarrow (\beta^{\pm}(n, m, k, x) \Rightarrow (b \vee B)[x/X]) \end{array}$$

This is the assertion we take as $w[\![c, B]\!]$ in this case. (In understanding this assertion compare it line-for-line with (2), bearing in mind that $\beta^{\pm}(n, m, i, x)$ means that x is the

ith element of the sequence encoded by the pair n, m.) The form of the assertion in the general case, for arbitrary l, is similar, though more clumsy, and left to the reader.

This completes the proof by structural induction. □

As **Assn** is expressive for any command c and assertion B there is an assertion $w[\![c, B]\!]$ with the property that

$$w[\![c, B]\!]^I = wp^I[\![c, B]\!]$$

for any interpretation I. Of course, the assertion $w[\![c, B]\!]$ constructed in the proof of expressiveness above, is not the unique assertion with this property (Why not?). However suppose A_0 is another assertion such that $A_0^I = wp^I[\![c, B]\!]$ for all I. Then

$$\models (w[\![c, B]\!] \iff A_0).$$

So the assertion expressing a weakest precondition is unique to within logical equivalence. The useful key fact about such an assertion $w[\![c, B]\!]$ is that, from the definition of weakest precondition, it is characterised by:

$$\sigma \models^I w[\![c, B]\!] \text{ iff } \mathcal{C}[\![c]\!]\sigma \models^I B,$$

for all states σ and interpretations I.

From the expressiveness of **Assn** we shall prove relative completeness. First an important lemma.

Lemma 7.6 *For $c \in$ **Com** and $B \in$ **Assn**, let $w[\![c, B]\!]$ be an assertion expressing the weakest precondition i.e. $w[\![c, B]\!]^I = wp^I[\![c, B]\!]$ (the assertion $w[\![c, B]\!]$ need not be necessarily that constructed by Theorem 7.5 above). Then*

$$\vdash \{w[\![c, B]\!]\}c\{B\}.$$

Proof: Let $w[\![c, B]\!]$ be an assertion which expresses the weakest precondition of a command c and postcondition B. We show by structural induction on c that

$$\vdash \{w[\![c, B]\!]\}c\{B\} \quad \text{for all } B \in \mathbf{Assn},$$

for all commands c.
(In all but the last case, the proof overlaps with that of Theorem 7.5.)

$c \equiv \mathbf{skip}$: In this case $\models w[\![\mathbf{skip}, B]\!] \iff B$, so $\vdash \{w[\![\mathbf{skip}, B]\!]\}\mathbf{skip}\{B\}$ by the consequence rule.

$c \equiv (X := a)$: In this case

$$\begin{aligned}\sigma \in wp^I[\![c, B]\!] \text{ iff } & \sigma[\mathcal{A}[\![a]\!]\sigma/X] \models^I B \\ \text{iff } & \sigma \models^I B[a/X].\end{aligned}$$

Thus $\models (w[\![c, B]\!] \iff B[a/X])$. Hence by the rule for assignment with the consequence rule we see $\vdash \{w[\![c, B]\!]\}c\{B\}$ in this case.

$c \equiv c_0; c_1$: In this case, for $\sigma \in \Sigma$ and interpretation I,

$$\begin{aligned}\sigma \models^I w[\![c_0; c_1, B]\!] \text{ iff } & \mathcal{C}[\![c_0; c_1]\!]\sigma \models^I B \\ \text{iff } & \mathcal{C}[\![c_1]\!](\mathcal{C}[\![c_0]\!]\sigma) \models^I B \\ \text{iff } & \mathcal{C}[\![c_0]\!]\sigma \models^I w[\![c_1, B]\!] \\ \text{iff } & \sigma \models^I w[\![c_0, w[\![c_1, B]\!]]\!].\end{aligned}$$

Thus $\models w[\![c_0; c_1, B]\!] \iff w[\![c_0, w[\![c_1, B]\!]]\!]$. By the induction hypothesis

$$\begin{aligned}&\vdash \{w[\![c_0, w[\![c_1, B]\!]]\!]\}c_0\{w[\![c_1, B]\!]\} \quad \text{and} \\ &\vdash \{w[\![c_1, B]\!]\}c_1\{B\}.\end{aligned}$$

Hence, by the rule for sequencing, we deduce

$$\vdash \{w[\![c_0, w[\![c_1, B]\!]]\!]\}c_0; c_1\{B\}$$

By the consequence rule we get

$$\vdash \{w[\![c_0; c_1, B]\!]\}c_0; c_1\{B\}.$$

$c \equiv \textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1$: In this case, for $\sigma \in \Sigma$ and interpretation I,

$$\begin{aligned}\sigma \models^I w[\![c, B]\!] \text{ iff } & \mathcal{C}[\![c]\!]\sigma \models^I B \\ \text{iff } & ([\mathcal{B}[\![b]\!]\sigma = \textbf{true} \ \&\ \mathcal{C}[\![c_0]\!]\sigma \models^I B] \text{ or} \\ & \quad [\mathcal{B}[\![b]\!]\sigma = \textbf{false} \ \&\ \mathcal{C}[\![c_1]\!]\sigma \models^I B]) \\ \text{iff } & ([\sigma \models^I b \ \&\ \sigma \models^I w[\![c_0, B]\!]] \text{ or} \\ & \quad [\sigma \models^I \neg b \ \&\ \sigma \models^I w[\![c_1, B]\!]]) \\ \text{iff } & \sigma \models^I [(b \wedge w[\![c_0, B]\!]) \vee (\neg b \wedge w[\![c_1, B]\!])].\end{aligned}$$

Hence

$$\models w[\![c, B]\!] \iff [(b \wedge w[\![c_0, B]\!]) \vee (\neg b \wedge w[\![c_1, B]\!])].$$

Now by the induction hypothesis

$$\vdash \{w[\![c_0, B]\!]\}c_0\{B\} \text{ and } \vdash \{w[\![c_1, B]\!]\}c_1\{B\}.$$

But

$$\models (w[\![c, B]\!] \wedge b) \iff w[\![c_0, B]\!] \quad \text{and}$$
$$\models (w[\![c, B]\!] \wedge \neg b) \iff w[\![c_1, B]\!].$$

So by the consequence rule

$$\vdash \{w[\![c, B]\!] \wedge b\}c_0\{B\} \text{ and } \vdash \{w[\![c, B]\!] \wedge \neg b\}c_1\{B\}.$$

By the rule for conditionals we obtain $\vdash \{w[\![c, B]\!]\}c\{B\}$ in this case.

Finally we consider the case:
$c \equiv$ **while** b **do** c_0 : Take $A \equiv w[\![c, B]\!]$. We show

(1) $\models \{A \wedge b\}c_0\{A\}$,
(2) $\models (A \wedge \neg b) \Rightarrow B$.

Then, from (1), by the induction hypothesis we obtain $\vdash \{A \wedge b\}c_0\{A\}$, so that by the while-rule $\vdash \{A\}c\{A \wedge \neg b\}$. Continuing, by (2), using the consequence rule, we obtain $\vdash \{A\}c\{B\}$. Now we prove (1) and (2).

(1) Let $\sigma \models^I A \wedge b$, for an interpretation I. Then $\sigma \models^I w[\![c, B]\!]$ and $\sigma \models^I b$, *i.e.* $\mathcal{C}[\![c]\!]\sigma \models^I B$ and $\sigma \models^I b$. But $\mathcal{C}[\![c]\!]$ is defined so

$$\mathcal{C}[\![c]\!] = \mathcal{C}[\![\textbf{if } b \textbf{ then } c_0; c \textbf{ else skip}]\!],$$

which makes $\mathcal{C}[\![c_0; c]\!]\sigma \models^I B$, *i.e.* $\mathcal{C}[\![c]\!](\mathcal{C}[\![c_0]\!]\sigma) \models^I B$. Therefore $\mathcal{C}[\![c_0]\!]\sigma \models^I w[\![c, B]\!]$, *i.e.* $\mathcal{C}[\![c_0]\!]\sigma \models^I A$. Thus $\models \{A \wedge b\}c_0\{A\}$.

(2) Let $\sigma \models^I A \wedge \neg b$, for an interpretation I. Then $\mathcal{C}[\![c]\!]\sigma \models^I B$ and $\sigma \models^I \neg b$. Again note $\mathcal{C}[\![c]\!] = \mathcal{C}[\![\textbf{if } b \textbf{ then } c_0; c \textbf{ else skip}]\!]$, so $\mathcal{C}[\![c]\!]\sigma = \sigma$. Therefore $\sigma \models^I B$. It follows that $\models^I A \wedge \neg b \Rightarrow B$. Thus $\models A \wedge \neg b \Rightarrow B$, proving (2).

This completes all the cases. Hence, by structural induction, the lemma is proved. □

Theorem 7.7 *The proof system for partial correctness is relatively complete, i.e. for any partial correctness assertion* $\{A\}c\{B\}$,

$$\vdash \{A\}c\{B\} \textit{ if } \models \{A\}c\{B\}.$$

Proof: Suppose $\models \{A\}c\{B\}$. Then by the above lemma $\vdash \{w[\![c, B]\!]\}c\{B\}$ where $w[\![c, B]\!]^I = wp^I[\![c, B]\!]$ for any interpretation I. Thus as $\models (A \Rightarrow w[\![c, B]\!])$, by the consequence rule, we obtain $\vdash \{A\}c\{B\}$. □

Exercise 7.8 (The Gödel β predicate)

(a) Let $n_0, \ldots, n_k$ be a sequence of natural numbers and let

$$m = (\max\{k, n_0, \ldots, n_k\})!$$

Show that the numbers

$$p_i = 1 + (1 + i) \times m, \text{ for } 0 \leq i \leq k$$

are coprime (*i.e.*, $\gcd(p_i, p_j) = 1$ for $i \neq j$) and that $n_i < p_i$.

(b) Further, define

$$c_i = p_0 \times \ldots \times p_k / p_i, \text{ for } 0 \leq i \leq k.$$

Show that for all i, $0 \leq i \leq k$, there is a unique d_i, $0 \leq d_i < p_i$, such that

$$(c_i \times d_i) \textbf{ mod } p_i = 1$$

(c) In addition, define

$$n = \sum_{i=0}^{k} c_i \times d_i \times n_i.$$

Show that

$$n_i = n \textbf{ mod } p_i$$

when $0 \leq i \leq k$.

(d) Finally prove lemma 3.

□

7.3 Proof of Gödel's Theorem

Gödel's Incompleteness Theorem amounts to the fact that the subset of valid assertions in **Assn** is not recursively enumerable (*i.e.*, there is no program which given assertions as input returns a confirmation precisely on the valid assertions—see the Appendix on computability for a precise definition and a more detailed treatment).

Theorem 7.9 *The subset of assertions* $\{A \in \textbf{Assn} \mid \models A\}$ *is not recursively enumerable.*

Proof: Suppose on the contrary that the set $\{A \in \mathbf{Assn} \mid \models A\}$ is recursively enumerable. Then there is a computable method to confirm that an assertion is valid. This provides a computable method to confirm that a command c diverges on the zero-state σ_0, in which each location X has contents 0:
Construct the assertion $w[\![c, \mathbf{false}]\!]$ as in the proof of Theorem 7.5. Let $\vec{X}$ consist of all the locations mentioned in $w[\![c, \mathbf{false}]\!]$. Let A be the assertion $w[\![c, \mathbf{false}]\!][\vec{0}/\vec{X}]$, obtained by replacing the locations by zeros. Then the divergence of c on the zero-state can be confirmed by checking the validity of A, for which there is assumed to be a computable method.
But it is known that the commands c which diverge on the zero-state do not form a recursively enumerable set—see Theorem A.12 in the Appendix. This contradiction shows $\{A \in \mathbf{Assn} \mid \models A\}$ to not be recursively enumerable. □

As a corollary we obtain Gödel's Incompleteness Theorem:

Theorem 7.10 *(Theorem 7.1 restated) (Gödel's Incompleteness Theorem):*
There is no effective proof system for **Assn** *such that its theorems coincide with the valid assertions of* **Assn**.

Proof: Assume there were an effective proof system such that for an assertion A, we have A is provable iff A is valid. The proof system being effective implies that there is a computable method to confirm precisely when something is a proof. Searching through all proofs systematically till a proof of an assertion A is found provides a computable method of confirming precisely when an assertion A is valid. Thus there cannot be an effective proof system. □

Although we have stated Gödel's Theorem for assertions **Assn** the presence of locations plays no essential role in the results. Gödel's Theorem is generally stated for the smaller language of assertions without locations—the language of arithmetic. The fact that the valid assertions in this language do not form a recursively enumerable set means that the axiomatisation of arithmetic is never finished—there will always be some fact about arithmetic which remains unprovable. Nor can we hope to have a program which generates an infinite list of axioms and effective proof rules so that all valid assertions about arithmetic follow. If there were such a program there would be an effective proof system for arithmetical assertions, contradicting Gödel's Incompleteness Theorem.

Gödel's result had tremendous historical significance. Gödel did not have the concepts of computability available to him. Rather his result stimulated logicians to research different formulations of what it meant to be computable. The original proof worked by expressing the concept of provability of a formal system for assertions as an assertion

itself, and constructing an assertion which was valid iff it was not provable. It should be admitted that we have only considered Gödel's First Incompleteness Theorem; there is also a second which says that a formal system for arithmetic cannot be proved free of contradiction in the system itself. It was clear to Gödel that his proofs of incompleteness hinged on being able to express a certain set of functions on the natural numbers by assertions—the set has come to be called the *primitive recursive functions*. The realisation that a simple extension led to a stable notion of computable function took some years longer, culminating in the Church-Turing thesis. The incompleteness theorem devastated the programme set up by Hilbert. As a reaction to paradoxes like Russell's in mathematical foundations, Hilbert had advocated a study of the finitistic methods employed when reasoning within some formal system, hoping that this would lead to proofs of consistency and completeness of important proof systems, like one for arithmetic. Gödel's Theorem established an absolute limit on the power of finitistic reasoning.

7.4 Verification conditions

In principle, the fact that **Assn** is expressive provides a method to reduce the demonstration that a partial correctness assertion is valid to showing the validity of an assertion in **Assn**; the validity of a partial correctness assertion of the form $\{A\}c\{B\}$ is equivalent to the validity of the assertion $A \Rightarrow w[\![c, B]\!]$, from which the command has been eliminated. In this way, given a theorem prover for predicate calculus we might hope to derive a theorem prover for **IMP** programs. Unfortunately, the method we used to obtain $w[\![c, B]\!]$ was convoluted and inefficient, and definitely not practical.

However, useful automated tools for establishing the validity of partial correctness assertions can be obtained along similar lines once we allow a little human guidance. Let us annotate programs by assertions. Define the syntactic set of annotated commands by:

$$c ::= \textbf{skip} \mid X := a \mid c_0; (X := a) \mid c_0; \{D\}c_1 \mid$$
$$\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1 \mid \textbf{while } b \textbf{ do } \{D\}c$$

where X is a location, a an arithmetic expression, b is a boolean expression, c, c_0, c_1 are annotated commands and D is an assertion such that in $c_0; \{D\}c_1$, the annotated command c_1, is *not* an assignment. The idea is that an assertion at a point in an annotated command is true whenever flow of control reaches that point. Thus we only annotate a command of the form $c_0; c_1$ at the point where control shifts from c_0 to c_1. It is unnecessary to do this when c_1 is an assignment $X := a$ because in that case an annotation can be derived simply from a postcondition. An annotated while-loop

$$\textbf{while } b \textbf{ do } \{D\}c$$

contains an assertion D which is intended to be an invariant.

An *annotated partial correctness assertion* has the form

$$\{A\}c\{B\}$$

where c is an annotated command. Annotated commands are associated with ordinary commands, got by ignoring the annotations. It is sometimes convenient to treat annotated commands as their associated commands. In this spirit, we say an annotated partial correctness assertion is valid when its associated (unannotated) partial correctness assertion is.

An annotated while-loop

$$\{A\}\textbf{while } b \textbf{ do } \{D\}c\{B\}$$

contains an assertion D, which we hope has been chosen judiciously so D is an invariant. Being an invariant means that

$$\{D \wedge b\}c\{D\}$$

is valid. In order to ensure

$$\{A\}\ \textbf{while } b \textbf{ do } \{D\}c\{B\}$$

is valid, once it is known that D is an invariant, it suffices to show that both assertions

$$A \Rightarrow D, \quad D \wedge \neg b \Rightarrow B$$

are valid. A quick way to see this is to notice that we can derive $\{A\}\textbf{while } b \textbf{ do } c\{B\}$ from $\{D \wedge b\}c\{D\}$ using the Hoare rules which we know to be sound. As is clear, not all annotated partial correctness assertions are valid. To be so it is sufficient to establish the validity of certain assertions, called *verification conditions* for which all mention of commands is eliminated. Define the verification conditions (abbreviated to vc) of an annotated partial correctness assertion by structural induction on annotated commands:

$$\begin{aligned}
vc(\{A\}\textbf{skip}\{B\}) &= \{A \Rightarrow B\} \\
vc(\{A\}X := a\{B\}) &= \{A \Rightarrow B[a/X]\} \\
vc(\{A\}c_0; X := a\{B\}) &= vc(\{A\}c_0\{B[a/X]\}) \\
vc(\{A\}c_0; \{D\}c_1\{B\}) &= vc(\{A\}c_0\{D\}) \cup vc(\{D\}c_1\{B\}) \\
&\quad \text{where } c_1 \text{is not an assignment} \\
vc(\{A\}\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1\{B\}) &= vc(\{A \wedge b\}c_0\{B\}) \cup vc(\{A \wedge \neg b\}c_1\{B\}) \\
vc(\{A\}\textbf{while } b \textbf{ do } \{D\}c\{B\}) &= vc(\{D \wedge b\}c\{D\}) \cup \{A \Rightarrow D\} \\
&\qquad \cup \{D \wedge \neg b \Rightarrow B\}
\end{aligned}$$

Exercise 7.11 Prove by structural induction on annotated commands that for all annotated partial correctness assertions $\{A\}c\{B\}$ if all assertions in $vc(\{A\}c\{B\})$ are valid then $\{A\}c\{B\}$ is valid. (The proof follows the general line of Lemma 7.6. A proof can be found in [42], Section 3.5.) □

Thus to show the validity of an annotated partial correctness assertion it is sufficient to show its verification conditions are valid. In this way the task of program verification can be passed to a theorem prover for predicate calculus. Some commercial program-verification systems, like Gypsy [41], work in this way.

Note, that while the validity of its verification conditions is sufficient to guarantee the validity of an annotated partial correctness assertion, it is not necessary. This can occur because the invariant chosen is inappropriate for the pre and post conditions. For example, although

$$\{\mathbf{true}\}\mathbf{while\ false\ do}\ \{\mathbf{false}\}\mathbf{skip}\{\mathbf{true}\}$$

is certainly valid with **false** as an invariant, its verification conditions contain

$$\mathbf{true} \Rightarrow \mathbf{false},$$

which is certainly not a valid assertion.

We conclude this section by pointing out a peculiarity in our treatment of annotated commands. Two commands, built up as $(c; X := a_1); X := a_2$ and $c; (X := a_1; X := a_2)$, are understood in essentially the same way; indeed in many imperative languages they would both be written as:

$$\begin{aligned} &c;\\ &X := a_1;\\ &X := a_2 \end{aligned}$$

However the two commands support different annotations according to our syntax of annotated commands. The first would only allow possible annotations to appear in c whereas the second would be annotated as $c; \{D\}(X := a_1; X := a_2)$. The rules for annotations do not put annotations before a single assignment but would put an annotation in before any other chain of assignments. This is even though it is still easily possible to derive the annotation from the postcondition, this time through a series of substitutions.

Exercise 7.12 Suggest a way to modify the syntax of annotated commands and the definition of their verification conditions to address this peculiarity, so that any chain of assignments or **skip** is treated in the same way as a single assignment is presently. □

Exercise 7.13 A larger project is to program a verification-condition generator (*e.g.*in standard ML or prolog) which, given an annotated partial correctness assertion as input, outputs a set, or list, of its verification conditions. (See Gordon's book [42] for a program in lisp.) □

7.5 Predicate transformers

This section is optional and presents an abstract, rather more mathematical view of assertions and weakest preconditions. Abstractly a command is a function $f : \Sigma \to \Sigma_\perp$ from states to states together with an element $\perp$, standing for undefined; such functions are sometimes called *state transformers*. They form a cpo, isomorphic to that of the partial functions on states, when ordered pointwise. Abstractly, an assertion for partial correctness is a subset of states which contains $\perp$, so we define the set of *partial correctness predicates* to be

$$\mathrm{Pred}(\Sigma) = \{Q \mid Q \subseteq \Sigma_\perp \ \& \ \perp \in Q\}.$$

We can make predicates into a cpo by ordering them by reverse inclusion. The cpo of predicates for partial correctness is

$$(\mathrm{Pred}(\Sigma), \supseteq).$$

Here, more information about the final state delivered by a command configuration corresponds to having bounded it to lie within a smaller set provided its execution halts. In particular the very least information corresponds to the element $\perp_{Pred} = \Sigma \cup \{\perp\}$. We shall use simply $\mathrm{Pred}(\Sigma)$ for the cpo of partial-correctness predicates.

The weakest precondition construction determines a continuous function on the cpo of predicates—a predicate transformer.[4]

Definition: Let $f : \Sigma \to \Sigma_\perp$ be a partial function on states. Define

$$\begin{aligned} &Wf : \mathrm{Pred}(\Sigma) \to \mathrm{Pred}(\Sigma); \\ &(Wf)(Q) = (f^{-1}Q) \cup \{\perp\} \\ &\textit{i.e.}, (Wf)(Q) = \{\sigma \in \Sigma_\perp \mid f(\sigma) \in Q\} \cup \{\perp\}. \end{aligned}$$

A command c can be taken to denote a state transformer $\mathcal{C}[\![c]\!] : \Sigma \to \Sigma_\perp$ with the convention that undefined is represented by $\perp$. Let B be an assertion. According to this understanding, with respect to an interpretation I,

$$(W(\mathcal{C}[\![c]\!]))(B^I) = wp^I[\![c, B]\!].$$

[4]This term is generally used for the corresponding notion when considering total correctness.

Exercise 7.14 Write ST for the cpo of state transformers $[\Sigma_\perp \to_\perp \Sigma_\perp]$ and PT for the cpo of predicate transformers $[\mathrm{Pred}(\Sigma) \to \mathrm{Pred}(\Sigma)]$.
Show $W : ST \to_\perp PT$ and W is continuous (Care! there are lots of things to check here).
Show $W(Id_{\Sigma_\perp}) = Id_{Pred(\Sigma)}$ *i.e.*, W takes the identity function on the cpo of states to the identity function on predicates $\mathrm{Pred}(\Sigma)$.
Show $W(f \circ g) = (Wg) \circ (Wf)$. □

In the context of total correctness Dijkstra has argued that one can specify the meaning of a command as a predicate transformer [36]. He argued that to understand a command amounts to knowing the weakest precondition which ensures a given postcondition. We do this for partial correctness. As we now have a cpo of predicates we also have the cpo

$$[\mathrm{Pred}(\Sigma) \to \mathrm{Pred}(\Sigma)]$$

of predicate transformers. Thus we can give a denotational semantics of commands in **IMP** as predicate transformers, instead of as state transformers. We can define a semantic function

$$\mathcal{P}t : \mathbf{Com} \to [\mathrm{Pred}(\Sigma) \to \mathrm{Pred}(\Sigma)]$$

from commands to predicate transformers. Although this denotational semantics, in which the denotation of a command is a predicate transformer is clearly a different denotational semantics to that using partial functions, if done correctly it should be equivalent in the sense that two commands denote the same predicate transformer iff they denote the same partial function. You may like to do this as the exercise below.

Exercise 7.15 (Denotations as predicate transformers)
Define a semantic function

$$\mathcal{P}t : \mathbf{Com} \to PT$$

by

$$\mathcal{P}t[\![X := a]\!]Q = \{\sigma \in \Sigma_\perp \mid \sigma[\mathcal{A}[\![a]\!]\sigma/X] \in Q\}$$
$$\mathcal{P}t[\![\mathbf{skip}]\!]Q = Q$$
$$\mathcal{P}t[\![c_0; c_1]\!]Q = \mathcal{P}t[\![c_0]\!](\mathcal{P}t[\![c_1]\!]Q)$$
$$\mathcal{P}t[\![\mathbf{if}\ b\ \mathbf{then}\ c_0\ \mathbf{else}\ c_1]\!]Q = \mathcal{P}t[\![c_0]\!](\overline{b} \cap Q) \cup \mathcal{P}t[\![c_1]\!](\overline{\neg b} \cap Q)$$
where $\overline{b} = \{\sigma \mid \sigma = \perp \text{ or } \mathcal{B}[\![b]\!]\sigma = \mathbf{true}\}$ for any boolean b
$$\mathcal{P}t[\![\mathbf{while}\ b\ \mathbf{do}\ c]\!]Q = \mathit{fix}(G)$$

where $G : PT \to PT$ is given by $G(p)(Q) = (\overline{b} \cap \mathcal{P}t[\![c_0]\!](p(Q)) \cup (\overline{\neg b} \cap Q)$.
Show G is continuous.

Show $W(\mathcal{C}[\![c]\!]_\perp) = Pt[\![c]\!]$ for any command c. Observe

$$Wf = Wf' \Rightarrow f = f'$$

for two strict continuous functions f, f' on $\Sigma_\perp$. Deduce

$$\mathcal{C}[\![c]\!] = \mathcal{C}[\![c']\!] \text{ iff } Pt[\![c]\!] = Pt[\![c']\!]$$

for any commands c, c'.

Recall the ordering on predicates. Because it is reverse inclusion:

$$fix(G) = \bigcap_{n\in\omega} G^n(\perp_{Pred}).$$

This suggests that if we were to allow infinite conjunctions in our language of assertions, and did not have quantifiers, we could express weakest preconditions directly. Indeed this is so, and you might like to extend **Bexp** by infinite conjunctions, to form another set of assertions to replace **Assn**, and modify the above semantics to give an assertion, of the new kind, which expresses the weakest precondition for each command. Once we have expressiveness a proof of relative completeness follows for this new kind of assertion, in the same way as earlier in Section 7.2. □

7.6 Further reading

The book "What is mathematical logic?" by Crossley *et al* [34] has an excellent explanation of Gödel's Incompleteness Theorem, though with the details missing. The logic texts by Kleene [54], Mendelson [61] and Enderton [38] have full treatments. A treatment aimed at Computer Science students is presented in the book [11] by Kfoury, Moll and Arbib. Cook's original proof of relative completeness in [33] used "strongest postconditions" instead of weakest preconditions; the latter are used instead by Clarke in [23] and his earlier work. The paper by Clarke has, in addition, some negative results showing the impossibility of having sound and relatively complete proof systems for programming languages richer than the one here. Apt's paper [8] provides good orientation. Alternative presentations of the material of this chapter can be found in [58], [13]. Gordon's book [42] contains a more elementary and detailed treatment of verification conditions.

Bibliography

[1] Abramsky, S., "The lazy lambda calculus." In Research Topics in Functional Programming (ed. Turner,D.A.), The UT Year of Programming Series, Addison-Wesley, 1990.

[2] Abramsky, S., "Domain theory in logical form." In IEEE Proc. of Symposium on Logic in Computer Science, 1987. Revised version in Annals of pure and Applied Logic, 51, 1991.

[3] Abramsky, S., "A computational interpretation of linear logic." To appear in Theoretical Computer Science.

[4] Aczel, P., "An introduction to inductive definitions." A chapter in the **Handbook of Mathematical Logic**, Barwise, J., (ed), North Holland, 1983.

[5] Alagić, S., and Arbib, M., "The design of well-structured and correct programs." Springer-Verlag, 1978.

[6] Andersen, H.R., "Model checking and boolean graphs." Proc. of ESOP 92, Springer-Verlag Lecture Notes in Computer Science vol.582, 1992.

[7] Andersen, H.R., "Local computation of alternating fixed-points." Tehnical Report No. 260, Computer Laboratory, University of Cambridge, 1992.

[8] Apt, K.R., "Ten years of Hoare's Logic: a survey." TOPLAS, 3, pp. 431-483, 1981.

[9] Apt, K.R, and Olderog, E-R., "**Verification of Sequential and Concurrent Programs**," Springer-Verlag, 1991.

[10] Arbib, M., and Manes, E., "**Arrows, structures and functors**." Academic Press, 1975.

[11] Kfoury, A.J., Moll, R.N. & Arbib, M.A., "**A programming approach to computability**." Springer-Verlag, 1982.

[12] Backhouse, R., "**Program construction and verification**." Prentice Hall, 1986.

[13] de Bakker, J., "**Mathematical theory of program correctness**." Prentice-Hall, 1980.

[14] Barendregt, H., "**The lambda calculus, its syntax and semantics**." North Holland, 1984.

[15] Barr, M., and Wells, C., "**Category theory for computer science**." Prentice-Hall, 1990.

[16] Berry, G., Curien, P-L., and Lévy, J-J., "Full abstraction for sequential languages: the state of the art. In Nivat, M., and Reynolds, J., (ed), Algebraic Methods in Semantics, Cambridge University Press, 1985.

[17] Sørensen, B.B., and Clausen, C., "Adequacy results for a lazy functional language with recursive and polymophic types." DAIMI Report, University of Aarhus, submitted to Theoretical Computer Science.

[18] Camilleri, J.A., and Winskel, G., "CCS with priority choice." Proc. of Symposium on Logic in Computer Science, Amsterdam, IEEE, 1991. Extended version to appear in Information and Computation.

[19] Crole, R., "Programming metalogics with a fixpoint type." University of Cambridge Computer Laboratory Technical Report No. 247, 1992.

[20] Cutland, N.J., "Computability: an introduction to recursive function theory." Cambridge University Press, 1983.

[21] Bird, R., "**Programs and machines**." John Wiley, 1976.

[22] Bird, R., and Wadler, P., "**Introduction to functional programming**." Prentice-Hall, 1988.

[23] Clarke, E.M. Jr., "The characterisation problem for Hoare Logics" in Hoare, C.A.R. and Shepherdson, J.C. (eds.), " Mathematical logic and programming languages." Prentice-Hall, 1985.

[24] Clarke, E.M., Emerson, E.A., and Sistla, A.P., "Automatic verification of finite state concurrent sytems using temporal logic." Proc. of 10th Annual ACM Symposium on Principles of Programming Languages, Austin, Texas, 1983.

[25] Cleaveland, R., "Tableau-based model checking in the propositional mu-calculus." Acta Informatica, 27, 1990.

[26] Clément, J., Despeyroux, J., Despeyroux, T., and Kahn, G., "A simple applicative language: mini-ML." Proc. of the 1986 ACM Conference on Lisp and Functional Programming, 1986.

[27] Cosmadakis, S.S., Meyer, A.R., and Riecke, J.G., "Completeness for typed lazy languages (Preliminary report)." Proc. of Symposium on Logic in Computer Science, Philadelphia, USA, IEEE, 1990.

[28] Despeyroux, J., "Proof of translation in natural semantics." Proc. of Symposium on Logic in Computer Science, Cambridge, Massachusetts, USA, IEEE, 1986.

[29] Despeyroux, T., "Typol: a formalism to implement natural semantics." INRIA Research Report 94, Roquencourt, France, 1988.

[30] Cleaveland, R., Parrow, J. and Steffen, B., "The Concurrency Workbench." Report of LFCS, Edinburgh University, 1988.
[31] Clocksin, W.F., and Mellish, C., "**Programming in PROLOG**." Springer-Verlag, 1981.
[32] Cohen, "**Programming for the 1990's**". Springer-Verlag, 1991.
[33] Cook, S.A., "Soundness and completeness of an axiom system for program verification." SIAM J. Comput. 7, pp. 70-90, 1978.
[34] Crossley, J.N., "**What is mathematical logic?**." Oxford University Press, 1972.
[35] Davis, M., "Hilbert's tenth problem is unsolvable." Am.Math.Monthly 80, 1973.
[36] Dijkstra, E.W., "**A discipline of programming**." Prentice-Hall, 1976.
[37] Emerson, A. and Lei, C., "Efficient model checking in fragments of the propositional mu-calculus." Proc. of Symposium on Logic in Computer Science, 1986.
[38] Enderton, H.B., "**A mathematical introduction to logic**." Academic Press, 1972.
[39] Enderton, H.B., "**Elements of set theory**." Academic Press, 1977.
[40] Girard, J-Y., Lafont, Y., and Taylor, P., "**Proofs and types**." Cambridge University Press, 1989.
[41] Good, D.I., "Mechanical proofs about computer programs." in Hoare, C.A.R., and Shepherdson, J.C. (eds.), "**Mathematical Logic and Programming Languages**." Prentice-Hall, 1985.
[42] Gordon, M.J.C., "**Programming language theory and its implementation**." Prentice-Hall, 1988.
[43] Gordon, M.J.C., HOL: A proof generating system for higher-order logic, in **VLSI Specification, Verification and Synthesis**, (ed. Birtwistle, G., and Subrahmanyam, P.A.) Kluwer, 1988.
[44] Gries, D., "**The science of programming.**" Springer Texts and Monographs in Computer Science, 1981.
[45] Hindley, R., and Seldin, J.P, "**Introduction to combinators and lambda-calculus**." Cambridge University Press, 1986.
[46] Godskesen, J.C., and Larsen, K.G., and Zeeberg, M., "TAV (Tools for Automatic Verification) users manual." Technical Report R 89-19, Department of Mathematics and Computer Science, Aalborg University, 1989. Presented at the workshop on Automated Methods for Finite State Systems, Grenoble, France, June 1989.
[47] Halmos, P.R., "**Naive set theory**." Litton Ed Publ. Inc., 1960.
[48] Hennessy, M.C, "**Algebraic theory of processes.**" MIT Press, 1988.
[49] Hoare, C.A.R., "Communicating sequential processes." CACM, vol.21, No.8, 1978.
[50] Hoare, C.A.R., "**Communicating sequential processes.**" Prentice-Hall, 1985.
[51] Huet, G., "A uniform approach to type theory." In **Logical Foundations of Functional Programming** (ed. Huet,G.), The UT Year of Programming Series, Addison-Wesley, 1990.
[52] Jensen, C.T., "The Concurrency Workbench with priorities." To appear in the proceedings of Computer Aided Verification, Aalborg, 1991, Springer-Verlag Lecture Notes in Computer Science.
[53] Johnstone, P.T., "**Stone spaces**." Cambridge University Press, 1982.
[54] Kleene, S.C., "**Mathematical logic**." John Wiley, 1967.
[55] Kozen, D., "Results on the propositional mu-calculus," Theoretical Computer Science 27, 1983.
[56] Lamport, L., "The temporal logic of actions." Technical Report 79, Digital Equipment Corporation, Systems Research Center, 1991.
[57] Larsen, K.G., "Proof systems for Hennessy-Milner logic." Proc. CAAP, 1988.
[58] Loeckx, J. and Sieber, K. "**The foundations of program verification**." John Wiley, 1984.
[59] Manna, Z., "**Mathematical theory of computation**." McGraw-Hill, 1974.
[60] Manna, Z., and Pnueli, A., "How to cook a temporal proof system for your pet language." Proc. of 10th Annual ACM Symposium on Principles of Programming Languages, Austin, Texas, 1983.
[61] Mendelson, E., "**Introduction to mathematical logic**." Van Nostrand, 1979.
[62] Milner, A.J.R.G., "Fully abstract models of typed lambda-calculi." Theoretical Computer Science 4, 1977.
[63] Milner, A.J.R.G., "**Communication and concurrency.**" Prentice Hall, 1989.
[64] Milner, A.J.R.G., "Operational and algebraic semantics of concurrent processes." A chapter in **Handbook of Theoretical Computer Science**, North Holland, 1990.
[65] Mitchell, J.C., "Type systems for programming languages." A chapter in **Handbook of Theoretical Computer Science**, North Holland, 1990.

[66] Moggi, E., "Categories of partial morphisms and the lambda$_p$-calculus." In proceedings of Category Theory and Computer Programming, Springer-Verlag Lecture Notes in Computer Science vol.240, 1986.
[67] Moggi, E., "Computational lambda-calculus and monads." Proc. of Symposium on Logic in Computer Science, Pacific Grove, California, USA, IEEE, 1989.
[68] Mosses, P.D., "Denotational semantics." A chapter in **Handbook of Theoretical Computer Science**, North Holland, 1990.
[69] Nielson. H.R., and Nielson, F., "**Semantics with applications: a formal introduction**." John Wiley, 1992.
[70] inmos, "**Occam programming manual**." Prentice Hall, 1984.
[71] Ong, C-H.L., "The lazy lambda calculus: an investigation into the foundations of functional programming." PhD thesis, Imperial College, University of London, 1988.
[72] Parrow, J., "Fairness properties in process algebra." PhD thesis, Uppsala University, Sweden, 1985.
[73] Paulson,L.C., "**ML for the working programmer**." Cambridge University Press, 1991.
[74] Paulson, L.C., "**Logic and computation: interactive proof with Cambridge LCF**." Cambridge University Press, 1987.
[75] Pitts, A., "Semantics of programming languages." Lecture notes, Computer Laboratory, University of Cambridge, 1989.
[76] Pitts, A., "A co-induction principle for recursively defined domains." University of Cambridge Computer Laboratory Technical Report No.252, 1992.
[77] Plotkin, G.D., "Call-by-name, Call-by-value and the lambda calculus." Theoretical Computer Science 1, 1975.
[78] Plotkin, G.D., "LCF considered as programming language." Theoretical Computer Science 5, 1977.
[79] Plotkin, G.D., "A powerdomain construction." SIAM J. Comput.5, 1976.
[80] Plotkin, G.D., "The Pisa lecture notes." Notes for lectures at the University of Edinburgh, extending lecture notes for the Pisa Summerschool, 1978.
[81] Plotkin, G.D., "Structural operational semantics." Lecture Notes, DAIMI FN-19, Aarhus University, Denmark, 1981 (reprinted 1991).
[82] Plotkin, G.D., "An operational semantics for CSP." In Formal Description of Programming Concepts II, Proc. of TC-2 Work. Conf. (ed. Bjørner, D.), North-Holland, 1982.
[83] Plotkin, G.D., "Types and partial functions." Notes of lectures at CSLI, Stanford University, 1985.
[84] Prawitz, D., "**Natural deduction, a proof-theoretical study**." Almqvist & Wiksell, Stockholm, 1965.
[85] Reisig, W., "**Petri nets: an introduction**." EATCS Monographs on Theoretical Computer Science, Springer-Verlag, 1985.
[86] Rogers, H., "**Theory of recursive functions and effective computability**." McGraw-Hill, 1967.
[87] Roscoe,A.W., and Reed,G.M., "**Domains for denotational semantics**." Prentice Hall, 1992.
[88] schmidt, D., "**Denotational semantics: a methodology for language development**." Allyn & Bacon, 1986.
[89] Scott, D.S., "Lectures on a mathematical theory of computation." PRG Report 19, Programming Research Group, Univ. of Oxford, 1980.
[90] Scott, D.S., "Domains for denotational semantics." In proceedings of ICALP '82, Springer-Verlag Lecture Notes in Computer Science vol.140, 1982.
[91] Scott, D.S., and Gunter, C., "Semantic domains." A chapter in **Handbook of Theoretical Computer Science**, North Holland, 1990.
[92] Smyth, M., "Powerdomains." JCSS 16(1), 1978.
[93] Stirling, C. and Walker D., "Local model checking the modal mu-calculus." Proc.of TAPSOFT, 1989.
[94] Stoughton, A., "**Fully abstract models of programming languages**." Pitman, 1988.
[95] Stoy, J., "**Denotational semantics: the Scott-Strachey approach to programming language theory**." MIT Press, 1977.
[96] Tarski, A., "A lattice-theoretical fixpoint theorem and its applications." Pacific Journal of Mathematics, 5, 1955.
[97] Tennent, R.D., "**Principles of programming languages**." Prentice-Hall, 1981.

[98] Vickers, S., "**Topology via logic**." Cambridge University Press, 1989.
[99] Vuillemin, J.E., "Proof techniques for recursive programs." PhD Thesis, Stanford Artificial Intelligence Laboratory, Memo AIM-218, 1973.
[100] Walker, D., "Automated analysis of mutual exclusion algorithms using CCS." Formal Aspects of Computing 1, 1989.
[101] Wikström, Å. "**Functional programming using Standard ML**." Prentice-Hall, 1987.
[102] Winskel, G., "On powerdomains and modality." Theoretical Computer Science 36, 1985.
[103] Winskel, G. and Larsen, K., "Using information systems to solve recursive domain equations effectively." In the proceedings of the conference on Abstract Datatypes, Sophia-Antipolis, France, Springer-Verlag Lecture Notes in Computer Science vol.173, 1984.
[104] Winskel, G., "Event structures." Lecture notes for the Advanced Course on Petri nets, September 1986, Springer-Verlag Lecture Notes in Computer Science, vol.255, 1987.
[105] Winskel, G., "An introduction to event structures." Lecture notes for the REX summerschool in temporal logic, May 88, Springer-Verlag Lecture Notes in Computer Science, vol.354, 1989.
[106] Winskel, G., "A note on model checking the modal nu-calculus." Theoretical Computer Science 83, 1991.
[107] Winskel, G., and Nielsen, M., "Models for concurrency." To appear as a chapter in the **Handbook of Logic and the Foundations of Computer Science**, Oxford University Press.
[108] Zhang, G-Q., "**Logic of domains**." Birkhäuser, 1991.